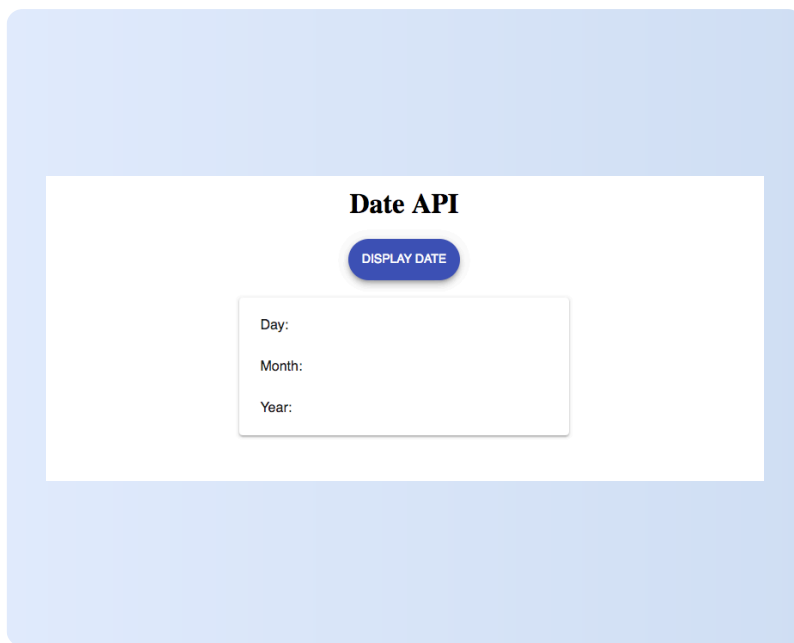


Question - 1**React: Date API**

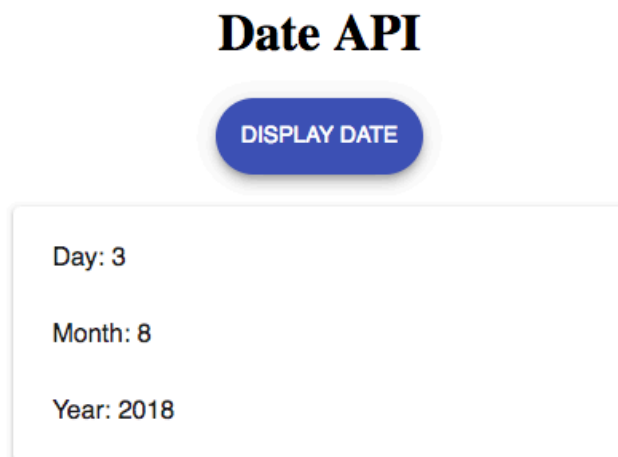
In the given single page React project, a button click prompts an HTTP GET request to an [API endpoint](#) that returns a *date* as its API response. Complete the project so it displays the current day, month, and year after clicking the button. Model the implementation after the instructions below. Certain core React functionalities have already been implemented. Complete the React application in order to pass all the unit tests.

Demo

- [Link to an animated GIF](#)
- Scroll sideways below to explore the screenshots. Clicking each screenshot opens it in a new window.



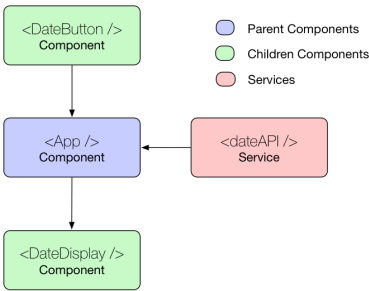
[Application's initial state before clicking the button.](#)



[Display date after clicking the button.](#)

Specifications

- Component flow diagram of the application is as follows:



▼ Framework Specific Instructions

- The project uses React 16 by default. Changing the React version may interfere with tests and is strictly discouraged. You may refer to the React 16 docs [here](#).
- The project uses [create-react-app](#) and [react-scripts](#) to automate serving and testing of the application.
- The project uses [enzyme](#) as a testing framework. Please refrain from changing the test framework or the tests themselves.
- The project uses React Material UI as a design framework. API docs are available [here](#) and can be used as a reference.

Question - 2
React: HackerBank

Create a banking application as shown below. Some core functionalities have already been implemented, but the application is not complete. Application requirements are given below, and the finished application must pass all of the unit tests.

[Hide animation](#) [Show animation](#)

H HackerBank

dd/mm/yyyy

Filter

Date	Description	Type	Amount (\$)	Available Balance
2019-12-01	THE HACKERUNIVERSITY DES: CCD+ ID:0000232343	Credit	1000	\$12,234.45
2019-11-25	HACKERBANK DES:DEBIT O ID: 0000987945787897987	Debit	2450.45	\$12,234.45
2019-11-29	HACKERBANK DES: CREDIT O ID:1223232323	Debit	999	\$10,928
2019-12-03	HACKERBANK INC. DES:CCD+ ID: 33375894749	Credit	1985.4	\$12,234.45
2019-11-29	HACKERBANK1 BP DES: MERCH PMT ID:1358570	Credit	1520.34	\$12,234.45
2019-11-29	HACKERBANK DES: DEBIT O ID:00097494729	Credit	564	\$12,234.45
2019-11-30	CREDIT CARD PAYMENT ID: 222349083	Debit	1987	\$12,234.45

Implement the following functionalities:

- All transactions are initially displayed inside the table in the order they are retrieved from the source. The source is passed down as 'txns' prop to TransactionTable component in App.js.
- Picking the date from the date input and pressing the 'Filter' button should display all the records for that date in the table. If no date is chosen, the 'Filter' button should not do anything.
- Clicking on the 'Amount (\$)' table header should sort the records in ascending order of amount. The behavior is the same for multiple clicks on 'Amount (\$)'.

Each transaction object contains the following properties :

- *String* date: The date when the transaction took place in the format YYYY-MM-DD.
- *String* description: The description of the transaction.
- *Number* type: The type of transaction, where 0 denotes a credit transaction and 1 denotes a debit transaction.
- *Float* amount: The total amount of the transaction.
- *String* balance: The balance of the account after the transaction was completed, prefixed with a dollar sign (\$).

```
{
  "date": "2019-12-03",
  "description": "HACKERBANK INC. DES:CCD+ ID: 33375894749",
  "type": 0,
  "amount": 1985.4,
  "balance": "$12,234.45"
}
```

Question - 3

React: Customer List

Create a customer list component as shown.

[Hide animation](#) [Show animation](#)



Customer List

Name

Add Customer

The component must have the following functionalities:

- The input should initially be empty.
- If no value is entered, clicking on the 'Add Customer' button should not do anything.
- Clicking on the 'Add Customer' button should add the input value to the list below. For this, add `<li>{input}</li>` to the `<ul data-testid="customer-list">` element.
- After the value is added to the list, it should clear the value in the input box.
- Please note that the customer list `<ul>` element should only be rendered if it has at least one customer added, i.e. at least one `<li>` child. When the app is mounted, since no customers are added, the `<ul>` element should not be rendered.
- All the values added by the button should be rendered in the list below.

The following *data-testid* attributes are required in the component for the tests to pass.

Component	Attribute
Input	<i>app-input</i>

Button	<i>submit-button</i>
Customer list	<i>customer-list</i>
List elements	<i>list-item0, list-item1, ...</i>

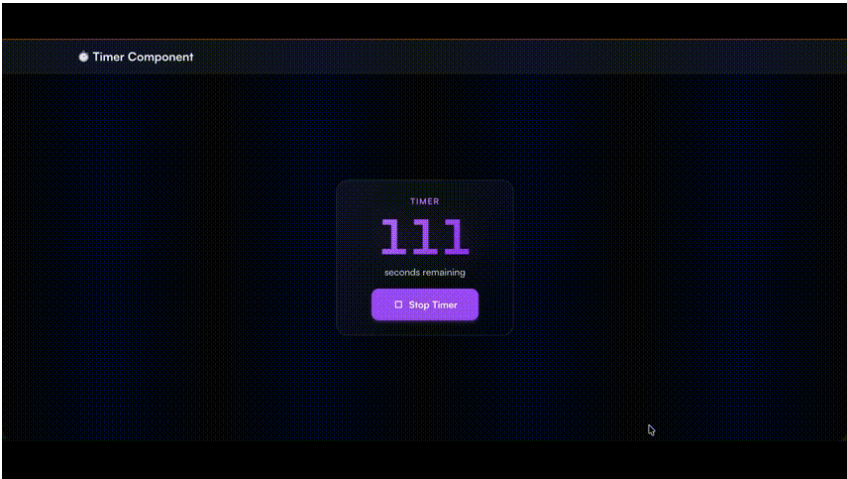
Please note that components have these *data-testid* attributes for test cases, and certain classes and ids for rendering purposes. They should not be changed.

Question - 4

React: Timer Component

Create a React Timer component that counts down from an initial value, updates every second, and allows the user to stop the timer manually.

[Hide animation](#)
[Show animation](#)



Your Users Should Be Able To:

- Pass an initial value to the component through a prop named initial.
- See the timer start automatically when the component mounts, decreasing by 1 every second.
 - Example: if initial = 100, after 1 second it shows 99, then 98, and so on.
- See the timer stop automatically when the value reaches 0.
- Click the Stop Timer button to stop the countdown at its current value.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
Title	<i>app-title</i>
Timer value	<i>timer-value</i>
Stop timer button	<i>stop-button</i>

Please note that components have these *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

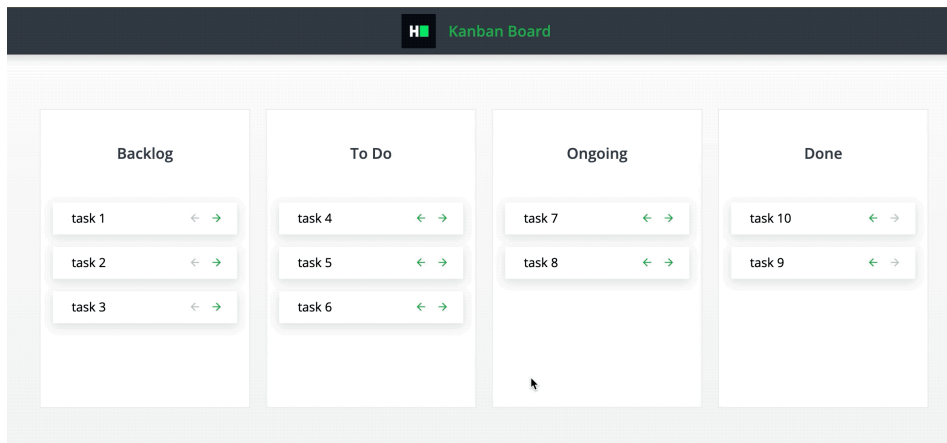
Question - 5

React: Kanban Board

Kanban is a popular workflow used in task management, project management, issue tracking, and other similar purposes. The workflow is usually visualized using a [Kanban Board](#).

Create a Kanban Board component with tasks, where each task consists of a name only, as shown below:

[Hide animation](#) [Show animation](#)



The component must have the following functionalities:

- The component board contains 4 stages of tasks in the sequence 'Backlog', 'To Do', 'Ongoing', and 'Done'.
- An array of tasks is passed as a prop to the component.
- In every individual stage, the tasks are rendered as a list `<ul>`, where each task is a single list item `<li>` that displays the name of the task.
- Each task list item has 2 icon buttons on the right:
 1. Back button: This moves the task to the previous stage in the sequence, if any. This button is disabled if the task is in the first stage.
 2. Forward button: This moves the task to the next stage in the sequence, if any. This button is disabled if the task is in the last stage.
- Each task has 2 properties:
 - name: The name of task. This is the unique identification for every task. [STRING]
 - stage: The stage of the task. [NUMBER] (0 represents the 'Backlog' stage, 1 represents the 'To Do' stage, 2 represents the 'Ongoing' stage, and 3 represents the 'Done' stage)

The following data-testid attributes are required in the component for the tests to pass:

- The `<ul>` for the 'Backlog' stage should have the data-testid attribute 'stage-0'.
- The `<ul>` for the 'To Do' stage should have the data-testid attribute 'stage-1'.
- The `<ul>` for the 'Ongoing' stage should have the data-testid attribute 'stage-2'.
- The `<ul>` for the 'Done' stage should have the data-testid attribute 'stage-3'.
- Every `<li>` task should follow these guidelines:
 1. The `<span>` containing the name should have the data-testid attribute 'TASK_NAME-name', where TASK_NAME is the name of the task joined by a hyphen symbol. For example, for the task named 'task 0', it should be 'task-0-name'. For the task named 'abc', it should be 'abc-name'.
 2. The back button should have the data-testid attribute 'TASK_NAME-back', where TASK_NAME is the name of the task joined by a hyphen symbol. For example, for the task named 'task 0', it should be 'task-0-back'. For the task named 'abc', it should be 'abc-back'.
 3. The forward button should have the data-testid attribute 'TASK_NAME-forward', where TASK_NAME is the name of the task joined by a hyphen symbol. For example, for the task named 'task 0', it should be 'task-0-forward'. For the task named 'abc', it should be 'abc-forward'.

Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.

Question - 1

React: FitTrack Pro

In this application, the task is to create a fitness tracker that allows users to log their daily activities, including exercise type, duration, and calories burned. Edit the *FitnessTracker.js*, *LogForm.js*, and *LogList.js* within the project structure to ensure the application functions as described below:

[Hide animation](#)

Functionality Requirements:

- Initial State of the project:
 - The exercise-type, duration, and calories-burned input fields should be empty initially.
 - The exercise-type input field should be of type text.
 - The duration and calories-burned input fields should be of type number.
 - The *Log Activity* button should not add any log until all fields have valid inputs.
- Logging functionality:
 - The *Log Activity* button should add a log only when all fields have valid values.
 - The duration and calories burned should be non-zero and positive numbers.
 - On clicking the *Log Activity* button:
 - Add a new entry in the log table, displaying the *exercise-type*, *duration* (in minutes), and *calories-burned*.
 - Clear the input fields after successfully adding the entry.
 - On clicking the *Reset Log* button:
 - All logs are cleared and reset to an empty list.
 - The error message should also be cleared.
- Log Display:
 - All log entries should appear within the application, under the *Activity Log* heading.
 - Each log entry, displays:
 - Exercise Type: the type of exercise logged
 - Duration: the time spent in minutes
 - Calories Burned: the calories burned during the activity
- Error Handling:
 - Display the error message, if the user tries to submit invalid data, such as empty fields or non-positive numbers.

- "Exercise type must not be empty." : If the exercise type input is empty.
- "Duration must be a positive number." : If the duration input is empty or non-positive.
- "Calories must be a positive number." : If the calories burned input is empty or non-positive.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

data-testid	Description
input-exerciseType	Input box to enter the type of exercise
input-duration	Input box to enter the duration of the activity (in minutes)
input-caloriesBurned	Input box to enter the calories burned
btn-logActivity	Button to log the activity
btn-resetLog	Button to reset all the activities
log-list	List of all the logs
log-entry	To display the user's logged activities
error-message	Error message for displaying the invalid inputs

Question - 2

React: Image Preview

Create a React web application called Image Preview that allows users to view, hide, and restore images interactively. The app should handle conditional rendering of images and placeholder components based on visibility state.

[Show animation](#)

The application has two components:

- The ImagePreview component that allows users to view and hide images by clicking on them.
- The HiddenImageDiv component that should be rendered when an image is to be hidden,

Your Users Should Be Able To

- View multiple images rendered from the provided images data prop.
- See each image displayed through the ImagePreview component, which conditionally renders:
- The image itself when its visible attribute is true.
- The HiddenImageDiv component when its visible attribute is false.
- Click on any visible image to hide it and display the HiddenImageDiv in its place.
- Click on a HiddenImageDiv to show the original image again.
- Click the Show All button to make all images visible and hide all HiddenImageDiv components.
- Click the Hide All button to hide all images and display only the HiddenImageDiv components.

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>images-div</i>	Images display div
<i>show-all-btn</i>	Show all images button
<i>hide-all-btn</i>	Hide all images button

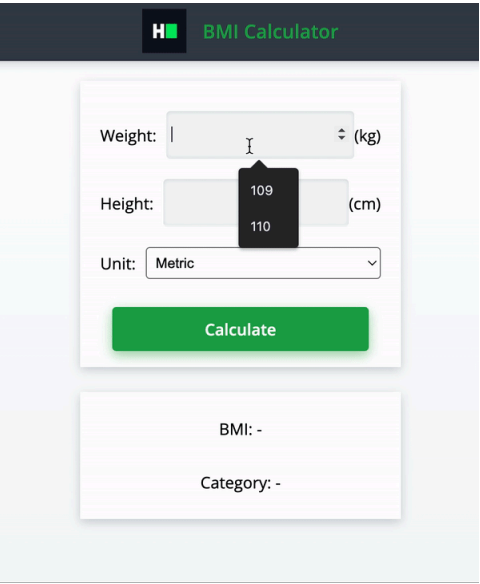
Note:

- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The only file that should be modified by the candidate is the `src/components/ImagePreview.js`.

Question - 3

React (Typescript): BMI Calculator

[Hide animation](#)



Using React and Typescript, design a Body Mass Index (BMI) calculator app. It should allow users to input their *weight* and *height*, then calculate and display their BMI. The app should work for both *Metric* and *Imperial* units, defined in type *Unit*. Certain core React functionalities are already implemented.

The application has one component: *BMICalculator/index.tsx* where all the functionalities should be implemented.

The component must have the following functionalities:

- Allow users to input their *weight* and *height*.
- Handle edge cases for *weight* and *height* input:
 - If the input is negative, set it to 0.
- Allow users to switch between *Metric* and *Imperial* units.
 - The default value for units should be *Metric*.
 - The default value for *weight* should be "".
 - The default value for *height* should be "".
 - The default value for BMI should be "-".
 - The default value for the category should be "-".
- If units are *Metric*:
 - *weight* should be in "kg"
 - *height* should be in "cm"
 - BMI should be calculated using this formula: $BMI = \text{weight (kg)} / ((\text{height (cm)} / 100) ^ 2)$
- If units are *Imperial*:
 - *weight* should be in "lbs"
 - *height* should be in "in"
 - BMI should be calculated using this formula: $BMI = (703 * \text{weight (lbs)}) / (\text{height (in)} ^ 2)$
- The BMI should be rounded to two decimal places.
- Display the BMI category according to the calculated BMI value:

BMI Category	BMI Range
Underweight	BMI < 18.5
Normal weight	18.5 ≤ BMI ≤ 24.9

Overweight	$25 \leq \text{BMI} \leq 29.9$
Obesity	$\text{BMI} \geq 30$

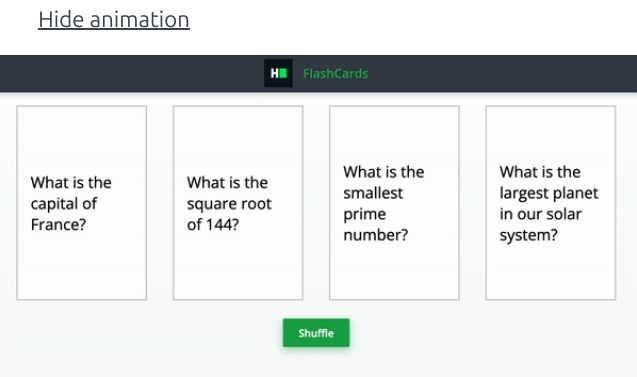
The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>bmi-calculator</i>	BMICalculator component
<i>weight-input</i>	Weight input field
<i>height-input</i>	Height input field
<i>unit-selector</i>	Unit selection dropdown
<i>calculate-button</i>	Calculate button
<i>bmi-result</i>	BMI result display
<i>bmi-category</i>	BMI category display

- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/BMICalculator/index.tsx*.

Question - 4

React (Typescript): Flashcards



Using Typescript and React, design a flashcard app that displays a series of flashcards with questions on the front and answers on the back. Certain core React functionalities are already implemented.

The application has two components: *FlashCardDeck.tsx* and *FlashCard.tsx* where the functionalities should be implemented.

- The component must have the following functionalities:
- Display a series of flashcards with questions on the front and answers on the back.
 - Clicking a flashcard should flip it to reveal the other side.
 - Update the *isFlipped* constant to a state variable in *FlashCard/index.tsx*:
 - When *isFlipped* is true
 - *flipped* is appended to divs with the class name *flashcard-content*.
 - the answer is shown.
 - When *isFlipped* is false
 - *"* is appended to divs with the class name *flashcard-content*.
 - the question is shown.

- Clicking the *Shuffle* button should reorder the flashcards.
 - Ensure the shuffled order of the flashcards is different from the original order present in *src/data/cards-data.ts*.
- All the types are defined under file *src/types/FlashCard.ts*

Create a type for *FlashCard* in *src/types/FlashCard.ts* with the following properties:

- *id*, a *number*
- *question*, a *string*
- *answer*, a *string*

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>flashcard-deck</i>	the main FlashCardApp component
<i>flashcard-container-{card.id}</i>	an individual FlashCard
<i>flashcard-question-{card.id}</i>	the question text in a FlashCard
<i>flashcard-answer-{card.id}</i>	the answer text in FlashCard
<i>shuffle-button</i>	a button to shuffle the flashcards

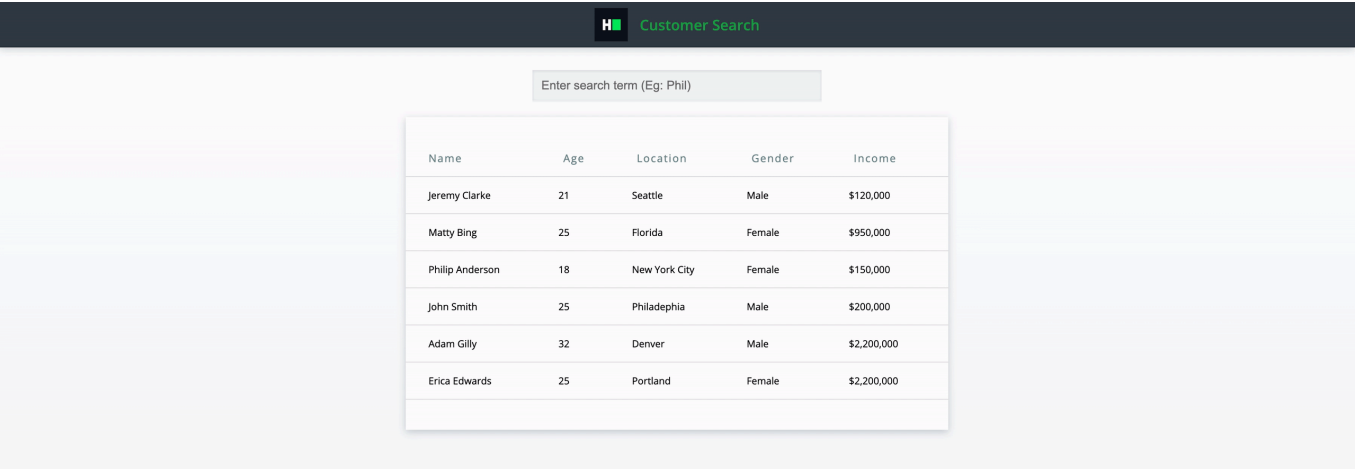
- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/FlashCardDeck.tsx*, *src/components/FlashCard.tsx*, and *src/types/FlashCard.ts*.

Question - 5

React: Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#)



The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
```

```
    income: "$120,000"
  }
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

Note:

- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The files that should be modified by the candidate are `src/components/CustomerList.js` and `src/components/SearchCustomer.js`. They are open by default in the system editor.
- Avoid making changes to other files in the project structure.

Question - 1

React: FitTrack Pro

In this application, the task is to create a fitness tracker that allows users to log their daily activities, including exercise type, duration, and calories burned. Edit the *FitnessTracker.js*, *LogForm.js*, and *LogList.js* within the project structure to ensure the application functions as described below:

[Hide animation](#)

Functionality Requirements:

- Initial State of the project:
 - The exercise-type, duration, and calories-burned input fields should be empty initially.
 - The exercise-type input field should be of type text.
 - The duration and calories-burned input fields should be of type number.
 - The *Log Activity* button should not add any log until all fields have valid inputs.
- Logging functionality:
 - The *Log Activity* button should add a log only when all fields have valid values.
 - The duration and calories burned should be non-zero and positive numbers.
 - On clicking the *Log Activity* button:
 - Add a new entry in the log table, displaying the *exercise-type*, *duration* (in minutes), and *calories-burned*.
 - Clear the input fields after successfully adding the entry.
 - On clicking the *Reset Log* button:
 - All logs are cleared and reset to an empty list.
 - The error message should also be cleared.
- Log Display:
 - All log entries should appear within the application, under the *Activity Log* heading.
 - Each log entry, displays:
 - Exercise Type: the type of exercise logged
 - Duration: the time spent in minutes
 - Calories Burned: the calories burned during the activity
- Error Handling:
 - Display the error message, if the user tries to submit invalid data, such as empty fields or non-positive numbers.

- "Exercise type must not be empty." : If the exercise type input is empty.
- "Duration must be a positive number." : If the duration input is empty or non-positive.
- "Calories must be a positive number." : If the calories burned input is empty or non-positive.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

data-testid	Description
input-exerciseType	Input box to enter the type of exercise
input-duration	Input box to enter the duration of the activity (in minutes)
input-caloriesBurned	Input box to enter the calories burned
btn-logActivity	Button to log the activity
btn-resetLog	Button to reset all the activities
log-list	List of all the logs
log-entry	To display the user's logged activities
error-message	Error message for displaying the invalid inputs

Question - 2

React: Image Preview

Create a React web application called Image Preview that allows users to view, hide, and restore images interactively. The app should handle conditional rendering of images and placeholder components based on visibility state.

[Show animation](#)

The application has two components:

- The ImagePreview component that allows users to view and hide images by clicking on them.
- The HiddenImageDiv component that should be rendered when an image is to be hidden,

Your Users Should Be Able To

- View multiple images rendered from the provided images data prop.
- See each image displayed through the ImagePreview component, which conditionally renders:
- The image itself when its visible attribute is true.
- The HiddenImageDiv component when its visible attribute is false.
- Click on any visible image to hide it and display the HiddenImageDiv in its place.
- Click on a HiddenImageDiv to show the original image again.
- Click the Show All button to make all images visible and hide all HiddenImageDiv components.
- Click the Hide All button to hide all images and display only the HiddenImageDiv components.

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>images-div</i>	Images display div
<i>show-all-btn</i>	Show all images button
<i>hide-all-btn</i>	Hide all images button

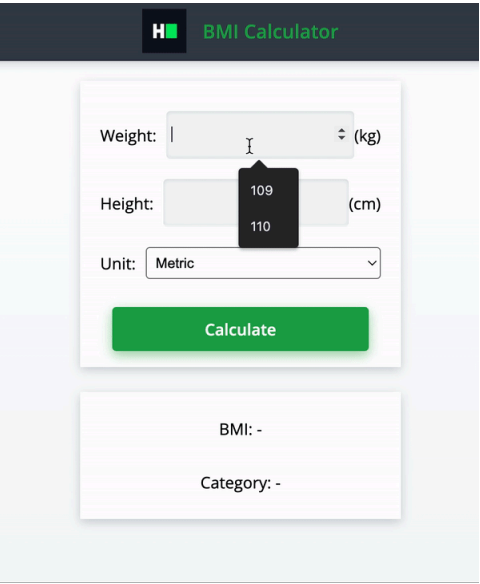
Note:

- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The only file that should be modified by the candidate is the `src/components/ImagePreview.js`.

Question - 3

React (Typescript): BMI Calculator

[Hide animation](#)



Using React and Typescript, design a Body Mass Index (BMI) calculator app. It should allow users to input their *weight* and *height*, then calculate and display their BMI. The app should work for both *Metric* and *Imperial* units, defined in type *Unit*. Certain core React functionalities are already implemented.

The application has one component: *BMICalculator/index.tsx* where all the functionalities should be implemented.

The component must have the following functionalities:

- Allow users to input their *weight* and *height*.
- Handle edge cases for *weight* and *height* input:
 - If the input is negative, set it to 0.
- Allow users to switch between *Metric* and *Imperial* units.
 - The default value for units should be *Metric*.
 - The default value for *weight* should be "".
 - The default value for *height* should be "".
 - The default value for BMI should be "-".
 - The default value for the category should be "-".
- If units are *Metric*:
 - *weight* should be in "kg"
 - *height* should be in "cm"
 - BMI should be calculated using this formula: $BMI = \text{weight (kg)} / ((\text{height (cm)} / 100) ^ 2)$
- If units are *Imperial*:
 - *weight* should be in "lbs"
 - *height* should be in "in"
 - BMI should be calculated using this formula: $BMI = (703 * \text{weight (lbs)}) / (\text{height (in)} ^ 2)$
- The BMI should be rounded to two decimal places.
- Display the BMI category according to the calculated BMI value:

BMI Category	BMI Range
Underweight	BMI < 18.5
Normal weight	18.5 ≤ BMI ≤ 24.9

Overweight	$25 \leq \text{BMI} \leq 29.9$
Obesity	$\text{BMI} \geq 30$

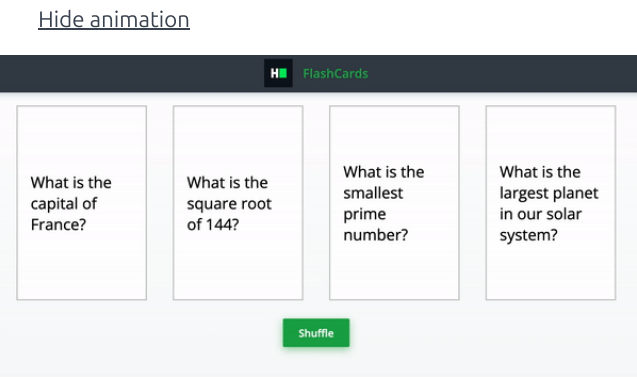
The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>bmi-calculator</i>	BMICalculator component
<i>weight-input</i>	Weight input field
<i>height-input</i>	Height input field
<i>unit-selector</i>	Unit selection dropdown
<i>calculate-button</i>	Calculate button
<i>bmi-result</i>	BMI result display
<i>bmi-category</i>	BMI category display

- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/BMICalculator/index.tsx*.

Question - 4

React (Typescript): Flashcards



Using Typescript and React, design a flashcard app that displays a series of flashcards with questions on the front and answers on the back. Certain core React functionalities are already implemented.

The application has two components: *FlashCardDeck.tsx* and *FlashCard.tsx* where the functionalities should be implemented.

- The component must have the following functionalities:
- Display a series of flashcards with questions on the front and answers on the back.
 - Clicking a flashcard should flip it to reveal the other side.
 - Update the *isFlipped* constant to a state variable in *FlashCard/index.tsx*:
 - When *isFlipped* is true
 - *flipped* is appended to divs with the class name *flashcard-content*.
 - the answer is shown.
 - When *isFlipped* is false
 - *"* is appended to divs with the class name *flashcard-content*.
 - the question is shown.

- Clicking the *Shuffle* button should reorder the flashcards.
 - Ensure the shuffled order of the flashcards is different from the original order present in *src/data/cards-data.ts*.
- All the types are defined under file *src/types/FlashCard.ts*

Create a type for *FlashCard* in *src/types/FlashCard.ts* with the following properties:

- *id*, a *number*
- *question*, a *string*
- *answer*, a *string*

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>flashcard-deck</i>	the main FlashCardApp component
<i>flashcard-container-{card.id}</i>	an individual FlashCard
<i>flashcard-question-{card.id}</i>	the question text in a FlashCard
<i>flashcard-answer-{card.id}</i>	the answer text in FlashCard
<i>shuffle-button</i>	a button to shuffle the flashcards

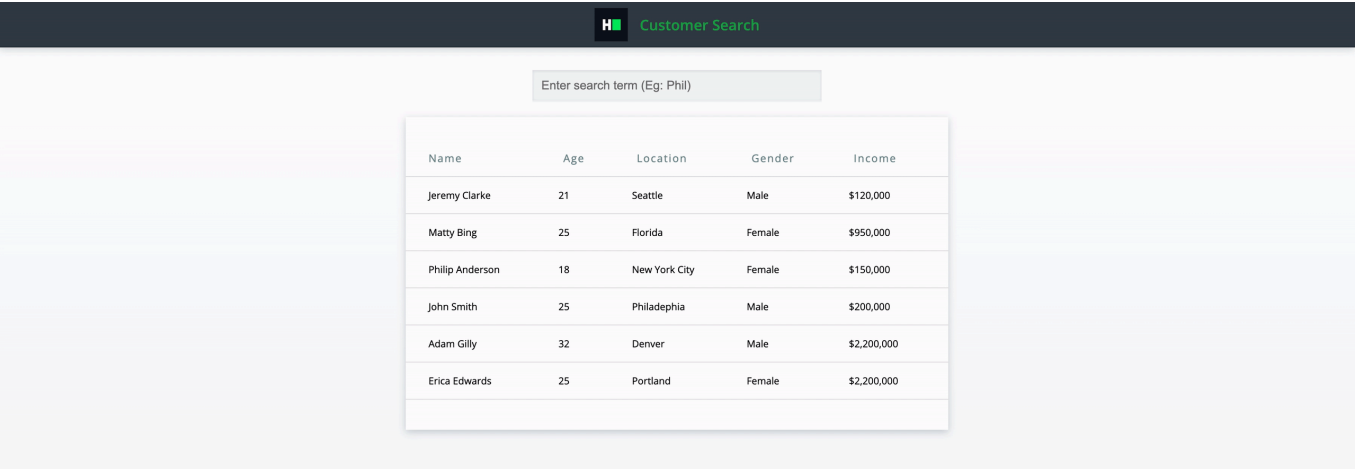
- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/FlashCardDeck.tsx*, *src/components/FlashCard.tsx*, and *src/types/FlashCard.ts*.

Question - 5

React: Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#)



The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
```

```
    income: "$120,000"
  }
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

Note:

- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The files that should be modified by the candidate are `src/components/CustomerList.js` and `src/components/SearchCustomer.js`. They are open by default in the system editor.
- Avoid making changes to other files in the project structure.

Question - 1

React: FitTrack Pro

In this application, the task is to create a fitness tracker that allows users to log their daily activities, including exercise type, duration, and calories burned. Edit the *FitnessTracker.js*, *LogForm.js*, and *LogList.js* within the project structure to ensure the application functions as described below:

[Hide animation](#)

Functionality Requirements:

- Initial State of the project:
 - The exercise-type, duration, and calories-burned input fields should be empty initially.
 - The exercise-type input field should be of type text.
 - The duration and calories-burned input fields should be of type number.
 - The *Log Activity* button should not add any log until all fields have valid inputs.
- Logging functionality:
 - The *Log Activity* button should add a log only when all fields have valid values.
 - The duration and calories burned should be non-zero and positive numbers.
 - On clicking the *Log Activity* button:
 - Add a new entry in the log table, displaying the *exercise-type*, *duration* (in minutes), and *calories-burned*.
 - Clear the input fields after successfully adding the entry.
 - On clicking the *Reset Log* button:
 - All logs are cleared and reset to an empty list.
 - The error message should also be cleared.
- Log Display:
 - All log entries should appear within the application, under the *Activity Log* heading.
 - Each log entry, displays:
 - Exercise Type: the type of exercise logged
 - Duration: the time spent in minutes
 - Calories Burned: the calories burned during the activity
- Error Handling:
 - Display the error message, if the user tries to submit invalid data, such as empty fields or non-positive numbers.

- "Exercise type must not be empty." : If the exercise type input is empty.
- "Duration must be a positive number." : If the duration input is empty or non-positive.
- "Calories must be a positive number." : If the calories burned input is empty or non-positive.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

data-testid	Description
input-exerciseType	Input box to enter the type of exercise
input-duration	Input box to enter the duration of the activity (in minutes)
input-caloriesBurned	Input box to enter the calories burned
btn-logActivity	Button to log the activity
btn-resetLog	Button to reset all the activities
log-list	List of all the logs
log-entry	To display the user's logged activities
error-message	Error message for displaying the invalid inputs

Question - 2

React: Image Preview

Create a React web application called Image Preview that allows users to view, hide, and restore images interactively. The app should handle conditional rendering of images and placeholder components based on visibility state.

[Show animation](#)

The application has two components:

- The ImagePreview component that allows users to view and hide images by clicking on them.
- The HiddenImageDiv component that should be rendered when an image is to be hidden,

Your Users Should Be Able To

- View multiple images rendered from the provided images data prop.
- See each image displayed through the ImagePreview component, which conditionally renders:
- The image itself when its visible attribute is true.
- The HiddenImageDiv component when its visible attribute is false.
- Click on any visible image to hide it and display the HiddenImageDiv in its place.
- Click on a HiddenImageDiv to show the original image again.
- Click the Show All button to make all images visible and hide all HiddenImageDiv components.
- Click the Hide All button to hide all images and display only the HiddenImageDiv components.

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>images-div</i>	Images display div
<i>show-all-btn</i>	Show all images button
<i>hide-all-btn</i>	Hide all images button

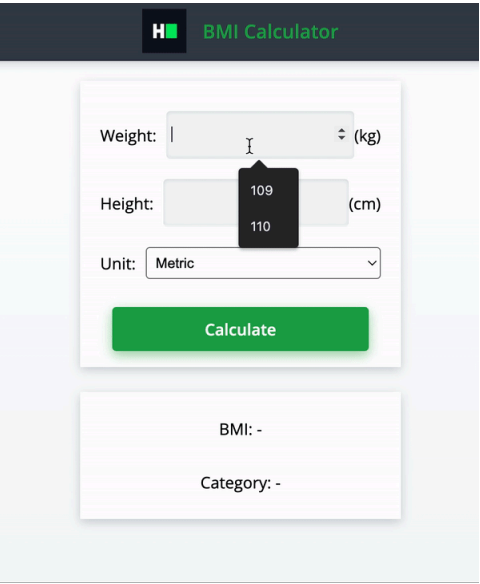
Note:

- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The only file that should be modified by the candidate is the `src/components/ImagePreview.js`.

Question - 3

React (Typescript): BMI Calculator

[Hide animation](#)



Using React and Typescript, design a Body Mass Index (BMI) calculator app. It should allow users to input their *weight* and *height*, then calculate and display their BMI. The app should work for both *Metric* and *Imperial* units, defined in type *Unit*. Certain core React functionalities are already implemented.

The application has one component: *BMICalculator/index.tsx* where all the functionalities should be implemented.

The component must have the following functionalities:

- Allow users to input their *weight* and *height*.
- Handle edge cases for *weight* and *height* input:
 - If the input is negative, set it to 0.
- Allow users to switch between *Metric* and *Imperial* units.
 - The default value for units should be *Metric*.
 - The default value for *weight* should be "".
 - The default value for *height* should be "".
 - The default value for BMI should be "-".
 - The default value for the category should be "-".
- If units are *Metric*:
 - *weight* should be in "kg"
 - *height* should be in "cm"
 - BMI should be calculated using this formula: $BMI = \text{weight (kg)} / ((\text{height (cm)} / 100) ^ 2)$
- If units are *Imperial*:
 - *weight* should be in "lbs"
 - *height* should be in "in"
 - BMI should be calculated using this formula: $BMI = (703 * \text{weight (lbs)}) / (\text{height (in)} ^ 2)$
- The BMI should be rounded to two decimal places.
- Display the BMI category according to the calculated BMI value:

BMI Category	BMI Range
Underweight	BMI < 18.5
Normal weight	18.5 ≤ BMI ≤ 24.9

Overweight	$25 \leq \text{BMI} \leq 29.9$
Obesity	$\text{BMI} \geq 30$

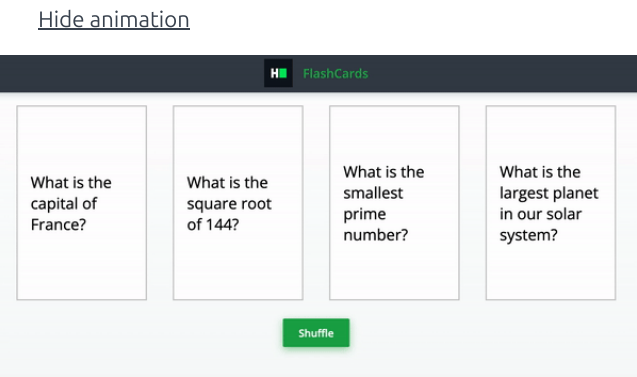
The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>bmi-calculator</i>	BMICalculator component
<i>weight-input</i>	Weight input field
<i>height-input</i>	Height input field
<i>unit-selector</i>	Unit selection dropdown
<i>calculate-button</i>	Calculate button
<i>bmi-result</i>	BMI result display
<i>bmi-category</i>	BMI category display

- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/BMICalculator/index.tsx*.

Question - 4

React (Typescript): Flashcards



Using Typescript and React, design a flashcard app that displays a series of flashcards with questions on the front and answers on the back. Certain core React functionalities are already implemented.

The application has two components: *FlashCardDeck.tsx* and *FlashCard.tsx* where the functionalities should be implemented.

- The component must have the following functionalities:
- Display a series of flashcards with questions on the front and answers on the back.
 - Clicking a flashcard should flip it to reveal the other side.
 - Update the *isFlipped* constant to a state variable in *FlashCard/index.tsx*:
 - When *isFlipped* is true
 - *flipped* is appended to divs with the class name *flashcard-content*.
 - the answer is shown.
 - When *isFlipped* is false
 - *"* is appended to divs with the class name *flashcard-content*.
 - the question is shown.

- Clicking the *Shuffle* button should reorder the flashcards.
 - Ensure the shuffled order of the flashcards is different from the original order present in *src/data/cards-data.ts*.
- All the types are defined under file *src/types/FlashCard.ts*

Create a type for *FlashCard* in *src/types/FlashCard.ts* with the following properties:

- *id*, a *number*
- *question*, a *string*
- *answer*, a *string*

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>flashcard-deck</i>	the main FlashCardApp component
<i>flashcard-container-{card.id}</i>	an individual FlashCard
<i>flashcard-question-{card.id}</i>	the question text in a FlashCard
<i>flashcard-answer-{card.id}</i>	the answer text in FlashCard
<i>shuffle-button</i>	a button to shuffle the flashcards

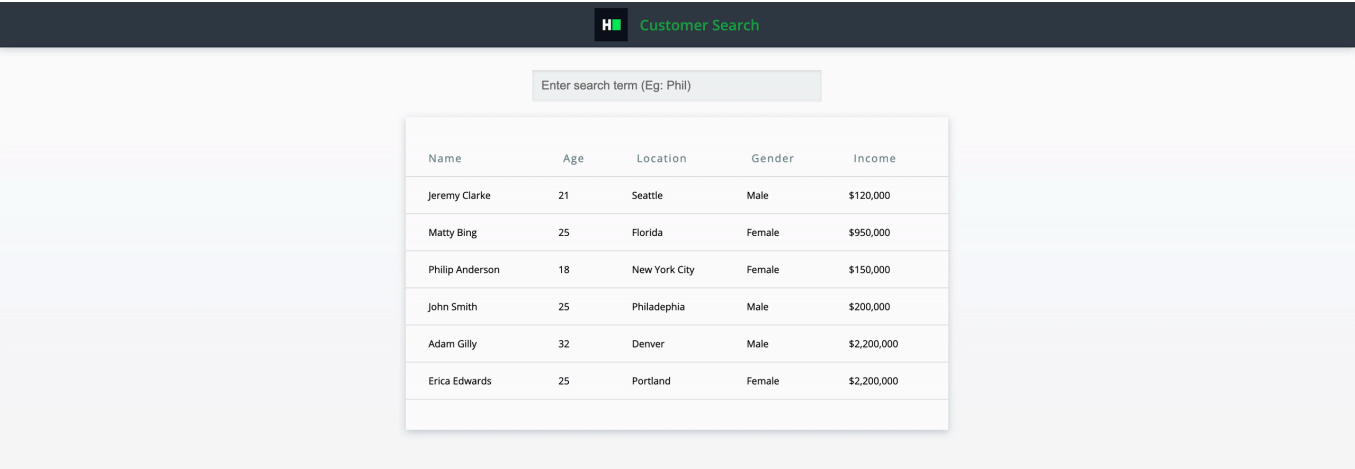
- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/FlashCardDeck.tsx*, *src/components/FlashCard.tsx*, and *src/types/FlashCard.ts*.

Question - 5

React: Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#)



The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
```

```
    income: "$120,000"
  }
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

Note:

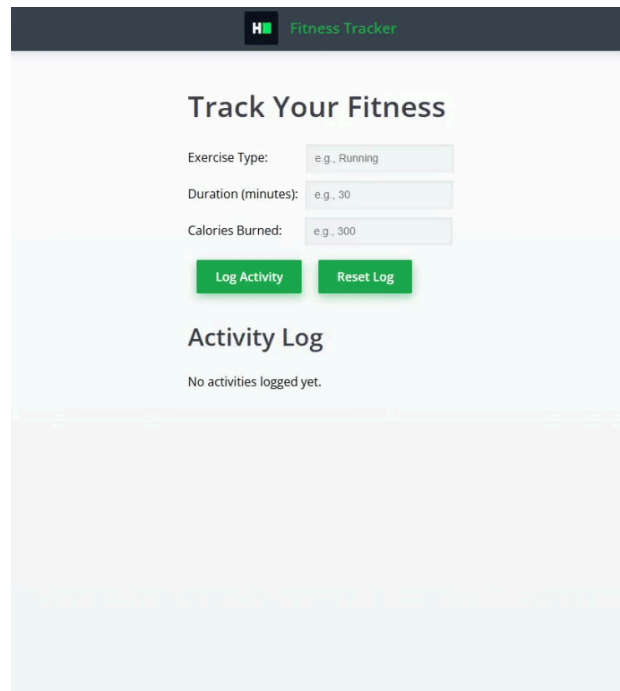
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The files that should be modified by the candidate are `src/components/CustomerList.js` and `src/components/SearchCustomer.js`. They are open by default in the system editor.
- Avoid making changes to other files in the project structure.

Question - 1

React: FitTrack Pro

In this application, the task is to create a fitness tracker that allows users to log their daily activities, including exercise type, duration, and calories burned. Edit the *FitnessTracker.js*, *LogForm.js*, and *LogList.js* within the project structure to ensure the application functions as described below:

[Hide animation](#)



Functionality Requirements:

- Initial State of the project:
 - The exercise-type, duration, and calories-burned input fields should be empty initially.
 - The exercise-type input field should be of type text.
 - The duration and calories-burned input fields should be of type number.
 - The *Log Activity* button should not add any log until all fields have valid inputs.
- Logging functionality:
 - The *Log Activity* button should add a log only when all fields have valid values.
 - The duration and calories burned should be non-zero and positive numbers.
 - On clicking the *Log Activity* button:
 - Add a new entry in the log table, displaying the *exercise-type*, *duration* (in minutes), and *calories-burned*.
 - Clear the input fields after successfully adding the entry.
 - On clicking the *Reset Log* button:
 - All logs are cleared and reset to an empty list.
 - The error message should also be cleared.
- Log Display:
 - All log entries should appear within the application, under the *Activity Log* heading.
 - Each log entry, displays:
 - Exercise Type: the type of exercise logged
 - Duration: the time spent in minutes
 - Calories Burned: the calories burned during the activity
- Error Handling:
 - Display the error message, if the user tries to submit invalid data, such as empty fields or non-positive numbers.

- "Exercise type must not be empty." : If the exercise type input is empty.
- "Duration must be a positive number." : If the duration input is empty or non-positive.
- "Calories must be a positive number." : If the calories burned input is empty or non-positive.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

data-testid	Description
input-exerciseType	Input box to enter the type of exercise
input-duration	Input box to enter the duration of the activity (in minutes)
input-caloriesBurned	Input box to enter the calories burned
btn-logActivity	Button to log the activity
btn-resetLog	Button to reset all the activities
log-list	List of all the logs
log-entry	To display the user's logged activities
error-message	Error message for displaying the invalid inputs

Question - 2

React: Image Preview

Create a React web application called Image Preview that allows users to view, hide, and restore images interactively. The app should handle conditional rendering of images and placeholder components based on visibility state.

[Show animation](#)

The application has two components:

- The ImagePreview component that allows users to view and hide images by clicking on them.
- The HiddenImageDiv component that should be rendered when an image is to be hidden,

Your Users Should Be Able To

- View multiple images rendered from the provided images data prop.
- See each image displayed through the ImagePreview component, which conditionally renders:
- The image itself when its visible attribute is true.
- The HiddenImageDiv component when its visible attribute is false.
- Click on any visible image to hide it and display the HiddenImageDiv in its place.
- Click on a HiddenImageDiv to show the original image again.
- Click the Show All button to make all images visible and hide all HiddenImageDiv components.
- Click the Hide All button to hide all images and display only the HiddenImageDiv components.

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>images-div</i>	Images display div
<i>show-all-btn</i>	Show all images button
<i>hide-all-btn</i>	Hide all images button

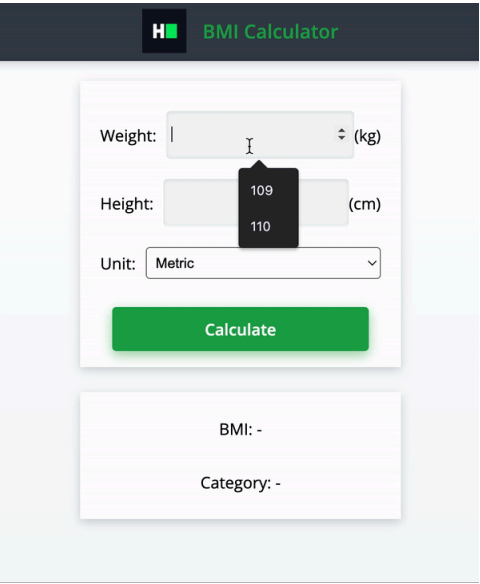
Note:

- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The only file that should be modified by the candidate is the `src/components/ImagePreview.js`.

Question - 3

React (Typescript): BMI Calculator

[Hide animation](#)



Using React and Typescript, design a Body Mass Index (BMI) calculator app. It should allow users to input their *weight* and *height*, then calculate and display their BMI. The app should work for both *Metric* and *Imperial* units, defined in type *Unit*. Certain core React functionalities are already implemented.

The application has one component: *BMICalculator/index.tsx* where all the functionalities should be implemented.

The component must have the following functionalities:

- Allow users to input their *weight* and *height*.
- Handle edge cases for *weight* and *height* input:
 - If the input is negative, set it to 0.
- Allow users to switch between *Metric* and *Imperial* units.
 - The default value for units should be *Metric*.
 - The default value for *weight* should be "".
 - The default value for *height* should be "".
 - The default value for BMI should be "-".
 - The default value for the category should be "-".
- If units are *Metric*:
 - *weight* should be in "kg"
 - *height* should be in "cm"
 - BMI should be calculated using this formula: $BMI = \text{weight (kg)} / ((\text{height (cm)} / 100) ^ 2)$
- If units are *Imperial*:
 - *weight* should be in "lbs"
 - *height* should be in "in"
 - BMI should be calculated using this formula: $BMI = (703 * \text{weight (lbs)}) / (\text{height (in)} ^ 2)$
- The BMI should be rounded to two decimal places.
- Display the BMI category according to the calculated BMI value:

BMI Category	BMI Range
Underweight	BMI < 18.5
Normal weight	18.5 ≤ BMI ≤ 24.9

Overweight	$25 \leq \text{BMI} \leq 29.9$
Obesity	$\text{BMI} \geq 30$

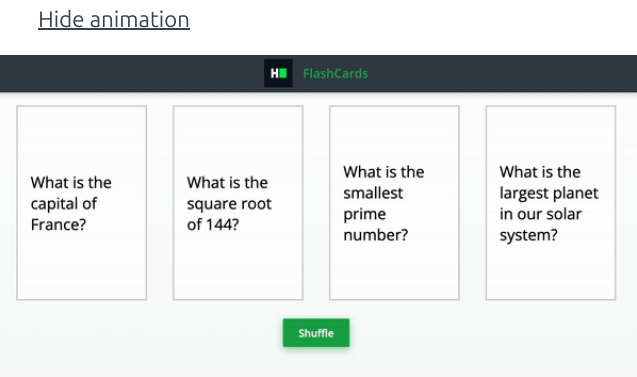
The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>bmi-calculator</i>	BMICalculator component
<i>weight-input</i>	Weight input field
<i>height-input</i>	Height input field
<i>unit-selector</i>	Unit selection dropdown
<i>calculate-button</i>	Calculate button
<i>bmi-result</i>	BMI result display
<i>bmi-category</i>	BMI category display

- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/BMICalculator/index.tsx*.

Question - 4

React (Typescript): Flashcards



Using Typescript and React, design a flashcard app that displays a series of flashcards with questions on the front and answers on the back. Certain core React functionalities are already implemented.

The application has two components: *FlashCardDeck.tsx* and *FlashCard.tsx* where the functionalities should be implemented.

- The component must have the following functionalities:
- Display a series of flashcards with questions on the front and answers on the back.
 - Clicking a flashcard should flip it to reveal the other side.
 - Update the *isFlipped* constant to a state variable in *FlashCard/index.tsx*:
 - When *isFlipped* is true
 - *flipped* is appended to divs with the class name *flashcard-content*.
 - the answer is shown.
 - When *isFlipped* is false
 - *"* is appended to divs with the class name *flashcard-content*.
 - the question is shown.

- Clicking the *Shuffle* button should reorder the flashcards.
 - Ensure the shuffled order of the flashcards is different from the original order present in *src/data/cards-data.ts*.
- All the types are defined under file *src/types/FlashCard.ts*

Create a type for *FlashCard* in *src/types/FlashCard.ts* with the following properties:

- *id*, a *number*
- *question*, a *string*
- *answer*, a *string*

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>flashcard-deck</i>	the main FlashCardApp component
<i>flashcard-container-{card.id}</i>	an individual FlashCard
<i>flashcard-question-{card.id}</i>	the question text in a FlashCard
<i>flashcard-answer-{card.id}</i>	the answer text in FlashCard
<i>shuffle-button</i>	a button to shuffle the flashcards

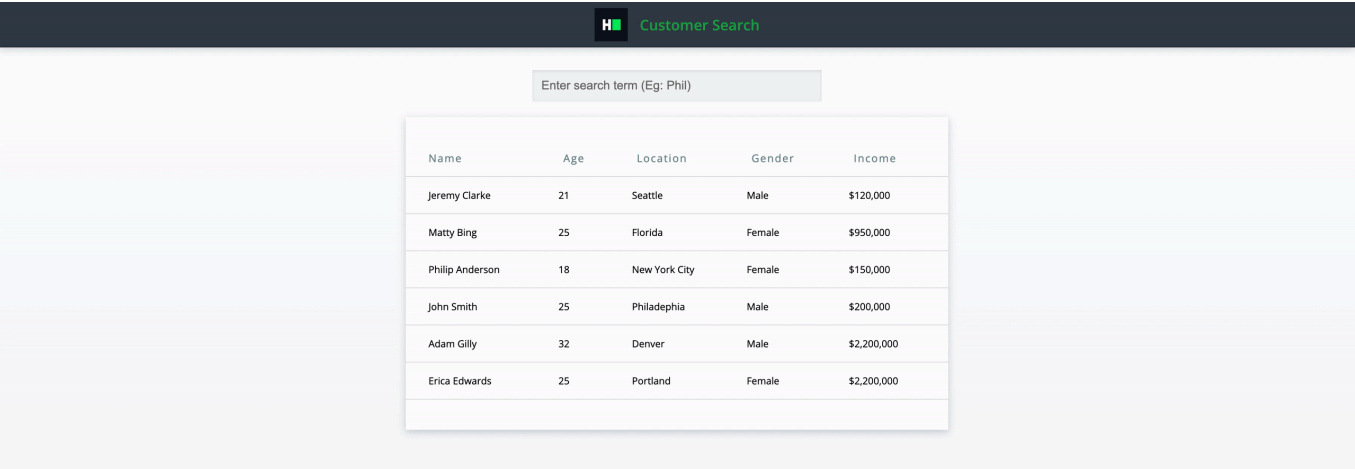
- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/FlashCardDeck.tsx*, *src/components/FlashCard.tsx*, and *src/types/FlashCard.ts*.

Question - 5

React: Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#)



The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
```

```
    income: "$120,000"
  }
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

Note:

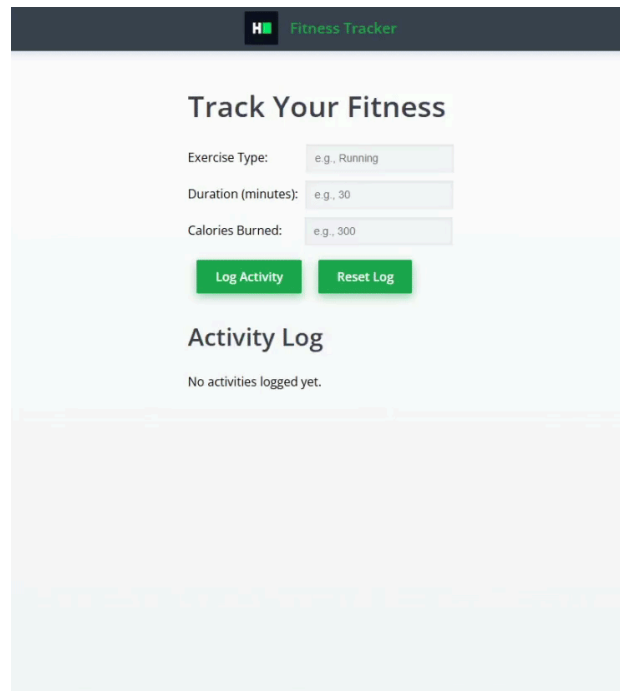
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The files that should be modified by the candidate are `src/components/CustomerList.js` and `src/components/SearchCustomer.js`. They are open by default in the system editor.
- Avoid making changes to other files in the project structure.

Question - 1

React: FitTrack Pro

In this application, the task is to create a fitness tracker that allows users to log their daily activities, including exercise type, duration, and calories burned. Edit the *FitnessTracker.js*, *LogForm.js*, and *LogList.js* within the project structure to ensure the application functions as described below:

[Hide animation](#)



The screenshot shows a web application titled "Fitness Tracker" with a dark header. The main content area has a light blue background and is titled "Track Your Fitness". It contains three input fields: "Exercise Type:" with a placeholder "e.g., Running", "Duration (minutes):" with a placeholder "e.g., 30", and "Calories Burned:" with a placeholder "e.g., 300". Below these fields are two green buttons: "Log Activity" and "Reset Log". Underneath the buttons is a section titled "Activity Log" with the text "No activities logged yet." and a table with a light blue header and a light blue body.

Functionality Requirements:

- Initial State of the project:
 - The exercise-type, duration, and calories-burned input fields should be empty initially.
 - The exercise-type input field should be of type text.
 - The duration and calories-burned input fields should be of type number.
 - The *Log Activity* button should not add any log until all fields have valid inputs.
- Logging functionality:
 - The *Log Activity* button should add a log only when all fields have valid values.
 - The duration and calories burned should be non-zero and positive numbers.
 - On clicking the *Log Activity* button:
 - Add a new entry in the log table, displaying the *exercise-type*, *duration* (in minutes), and *calories-burned*.
 - Clear the input fields after successfully adding the entry.
 - On clicking the *Reset Log* button:
 - All logs are cleared and reset to an empty list.
 - The error message should also be cleared.
- Log Display:
 - All log entries should appear within the application, under the *Activity Log* heading.
 - Each log entry, displays:
 - Exercise Type: the type of exercise logged
 - Duration: the time spent in minutes
 - Calories Burned: the calories burned during the activity
- Error Handling:
 - Display the error message, if the user tries to submit invalid data, such as empty fields or non-positive numbers.

- "Exercise type must not be empty." : If the exercise type input is empty.
- "Duration must be a positive number." : If the duration input is empty or non-positive.
- "Calories must be a positive number." : If the calories burned input is empty or non-positive.

Note:

The following data-testid attributes are required in the components for the test cases to pass, do not change/delete them:

Table of data-testid

data-testid	Description
input-exerciseType	Input box to enter the type of exercise
input-duration	Input box to enter the duration of the activity (in minutes)
input-caloriesBurned	Input box to enter the calories burned
btn-logActivity	Button to log the activity
btn-resetLog	Button to reset all the activities
log-list	List of all the logs
log-entry	To display the user's logged activities
error-message	Error message for displaying the invalid inputs

Question - 2

React: Image Preview

Create a React web application called Image Preview that allows users to view, hide, and restore images interactively. The app should handle conditional rendering of images and placeholder components based on visibility state.

[Show animation](#)

The application has two components:

- The ImagePreview component that allows users to view and hide images by clicking on them.
- The HiddenImageDiv component that should be rendered when an image is to be hidden,

Your Users Should Be Able To

- View multiple images rendered from the provided images data prop.
- See each image displayed through the ImagePreview component, which conditionally renders:
- The image itself when its visible attribute is true.
- The HiddenImageDiv component when its visible attribute is false.
- Click on any visible image to hide it and display the HiddenImageDiv in its place.
- Click on a HiddenImageDiv to show the original image again.
- Click the Show All button to make all images visible and hide all HiddenImageDiv components.
- Click the Hide All button to hide all images and display only the HiddenImageDiv components.

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>images-div</i>	Images display div
<i>show-all-btn</i>	Show all images button
<i>hide-all-btn</i>	Hide all images button

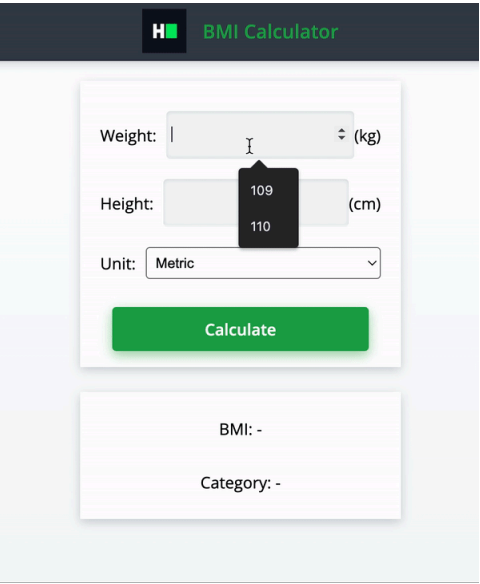
Note:

- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The only file that should be modified by the candidate is the `src/components/ImagePreview.js`.

Question - 3

React (Typescript): BMI Calculator

[Hide animation](#)



Using React and Typescript, design a Body Mass Index (BMI) calculator app. It should allow users to input their *weight* and *height*, then calculate and display their BMI. The app should work for both *Metric* and *Imperial* units, defined in type *Unit*. Certain core React functionalities are already implemented.

The application has one component: *BMICalculator/index.tsx* where all the functionalities should be implemented.

The component must have the following functionalities:

- Allow users to input their *weight* and *height*.
- Handle edge cases for *weight* and *height* input:
 - If the input is negative, set it to 0.
- Allow users to switch between *Metric* and *Imperial* units.
 - The default value for units should be *Metric*.
 - The default value for *weight* should be "".
 - The default value for *height* should be "".
 - The default value for BMI should be "-".
 - The default value for the category should be "-".
- If units are *Metric*:
 - *weight* should be in "kg"
 - *height* should be in "cm"
 - BMI should be calculated using this formula: $BMI = \text{weight (kg)} / ((\text{height (cm)} / 100) ^ 2)$
- If units are *Imperial*:
 - *weight* should be in "lbs"
 - *height* should be in "in"
 - BMI should be calculated using this formula: $BMI = (703 * \text{weight (lbs)}) / (\text{height (in)} ^ 2)$
- The BMI should be rounded to two decimal places.
- Display the BMI category according to the calculated BMI value:

BMI Category	BMI Range
Underweight	BMI < 18.5
Normal weight	18.5 ≤ BMI ≤ 24.9

Overweight	$25 \leq \text{BMI} \leq 29.9$
Obesity	$\text{BMI} \geq 30$

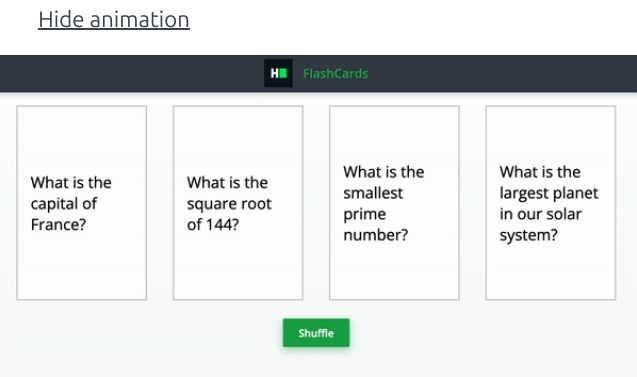
The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>bmi-calculator</i>	BMICalculator component
<i>weight-input</i>	Weight input field
<i>height-input</i>	Height input field
<i>unit-selector</i>	Unit selection dropdown
<i>calculate-button</i>	Calculate button
<i>bmi-result</i>	BMI result display
<i>bmi-category</i>	BMI category display

- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/BMICalculator/index.tsx*.

Question - 4

React (Typescript): Flashcards



Using Typescript and React, design a flashcard app that displays a series of flashcards with questions on the front and answers on the back. Certain core React functionalities are already implemented.

The application has two components: *FlashCardDeck.tsx* and *FlashCard.tsx* where the functionalities should be implemented.

- The component must have the following functionalities:
- Display a series of flashcards with questions on the front and answers on the back.
 - Clicking a flashcard should flip it to reveal the other side.
 - Update the *isFlipped* constant to a state variable in *FlashCard/index.tsx*:
 - When *isFlipped* is true
 - *flipped* is appended to divs with the class name *flashcard-content*.
 - the answer is shown.
 - When *isFlipped* is false
 - *"* is appended to divs with the class name *flashcard-content*.
 - the question is shown.

- Clicking the *Shuffle* button should reorder the flashcards.
 - Ensure the shuffled order of the flashcards is different from the original order present in *src/data/cards-data.ts*.
- All the types are defined under file *src/types/FlashCard.ts*

Create a type for *FlashCard* in *src/types/FlashCard.ts* with the following properties:

- *id*, a *number*
- *question*, a *string*
- *answer*, a *string*

The following *data-testid* attributes are required in the components for the tests to pass:

Attribute	Component
<i>flashcard-deck</i>	the main FlashCardApp component
<i>flashcard-container-{card.id}</i>	an individual FlashCard
<i>flashcard-question-{card.id}</i>	the question text in a FlashCard
<i>flashcard-answer-{card.id}</i>	the answer text in FlashCard
<i>shuffle-button</i>	a button to shuffle the flashcards

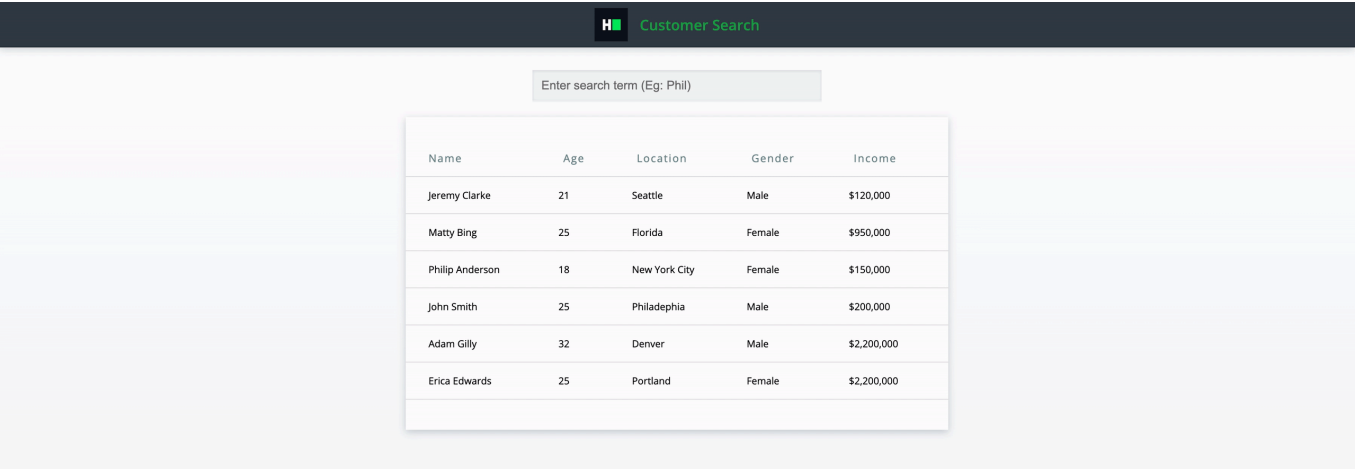
- Note:
- Components have *data-testid* attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are *src/components/FlashCardDeck.tsx*, *src/components/FlashCard.tsx*, and *src/types/FlashCard.ts*.

Question - 5

React: Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#)



The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
```

```
    income: "$120,000"
  }
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

Note:

- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
- The files that should be modified by the candidate are `src/components/CustomerList.js` and `src/components/SearchCustomer.js`. They are open by default in the system editor.
- Avoid making changes to other files in the project structure.

Question - 1

React(TypeScript): Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)

 Customer Search

Name	Age	Location	Gender	Income
Jeremy Clarke	21	Seattle	Male	\$120,000
Matty Bing	25	Florida	Female	\$950,000
Philip Anderson	18	New York City	Female	\$150,000
John Smith	25	Philadelphia	Male	\$200,000
Adam Gilly	32	Denver	Male	\$2,200,000
Erica Edwards	25	Portland	Female	\$2,200,000

The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
  income: "$120,000"
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
-----------	-----------

search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

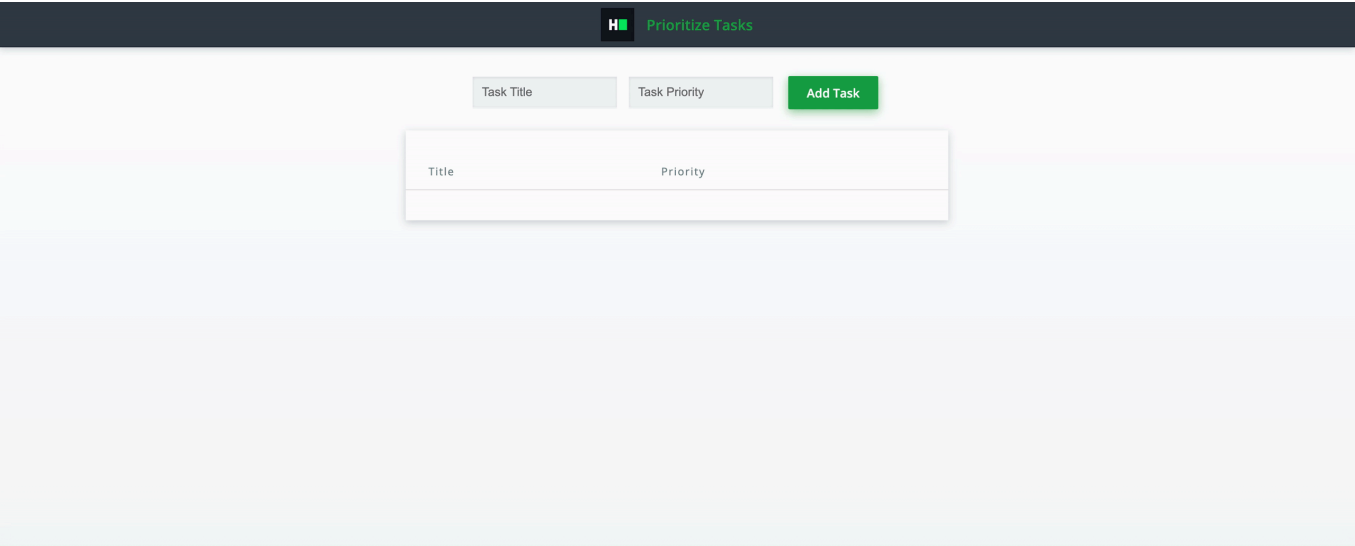
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are `src/components/CustomerList.tsx` and `src/components/SearchCustomer.tsx`. They are open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 2

React(TypeScript): Prioritize Tasks

Complete the component as shown to pass the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only one component, `Tasks`, where all the functionalities will be implemented.

- The component must have the following functionalities:
- The type of input for the:
 - Task Name input box should be "text".
 - Task Priority input box should be "number".
 - The initial view should have no tasks in the table.
 - Clicking the `Add` button should:
 - add a task and its priority in the table and sort the tasks according to priority.
 - display an alert saying "Please enter both title and task priority" if any input boxes are empty.
 - reset the input boxes to empty after adding a valid task to the table.
 - Clicking the `Delete` button should delete the corresponding task from the table.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
input-task-name	Input box for task name
input-task-priority	Input box for task priority

submit-button	Button for adding task
tasksList	List of tasks

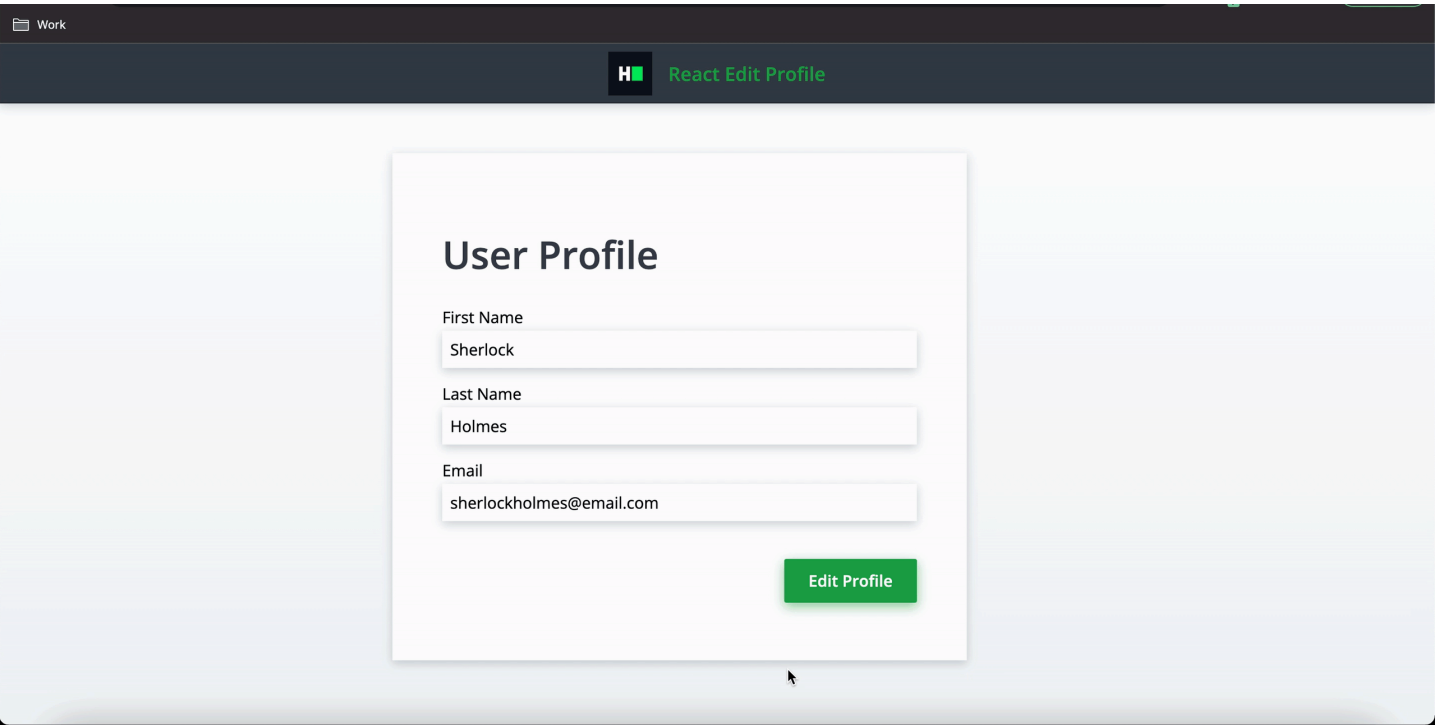
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The file that should be modified by the candidate `src/components/Books/index.tsx` is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 3

React(TypeScript): Edit Profile

Complete a React Edit Profile application as shown in order to pass all the unit tests. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only 1 component:

- The Profile component enables the user to update his/her details.

The application has the following functionalities:

- The Profile component shows the user their details and allows editing.
- Initially, there should be 3 divs with the following values:
 - First Name: **Sherlock**
 - Last Name: **Holmes**
 - Email: **sherlockholmes@email.com**
- Initially, the button should have the text value Edit Profile.
- The following functionality should be implemented when the user clicks the 'Edit Profile' button.
 - The user can now edit the details in the divs.
 - The button should now have the text value "Save Changes".
- The following functionality should be implemented when the user edits the details and clicks the Save Changes button.
 - If one or more of the fields is empty, there should be an alert prompt with the text "Please enter all profile fields" and no values should be updated.
 - If all the fields are non-empty, there should be an alert prompt with the text "Profile updated successfully".
 - The input values should be the updated values and should be in uneditable divs.
 - The button text should be Edit Profile.

The following data-testid attributes are required in the component for the tests to pass:

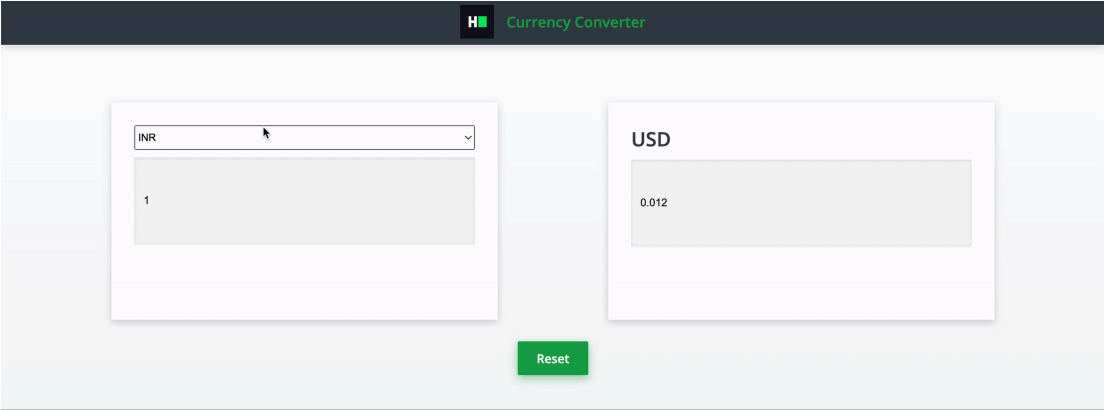
Description	Name
First Name <div>	firstNameDiv
Last Name <div>	lastNameDiv
Email <div>	emailDiv
First Name <input>	firstNameInput
Last Name <input>	lastNameInput
Email <input>	emailInput
<button>.	changeButton

The component has data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 4
React(TypeScript): Currency Converter

Complete the currency converter component that converts the value in any of the given currencies to USD as shown.

[Hide animation](#) [Show animation](#)



The list of available currencies is imported as "exchangeRates" in the component that can be used to render the list of currencies. The initial values for the currency and the input box are also imported from "exchangeRates.ts".

It is given that:

- The first input box is to be used to take input.
- The second input box is read-only to display the converted value.
- Only positive integer or decimal values can be taken as input.

The component must have the following functionalities:

- The default value for:
 - input box should be 1.
 - the currency to display in the select menu should be INR.
 - The second input box should be the conversion of 1 INR to USD i.e. 0.012.
- The select menu should have five currencies in the options.

- The conversion should happen automatically:
 - when there is any input change.
 - on selecting another currency.
- The converted value should be up to 3 decimal places.
- The placeholder text should change automatically according to the currency.
- The Reset button should reset the input to 1 and give the conversion in the current currency.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
select menu	select-currency
first input box	input-value
second input box	converted-value
Clear Input button	clear-value

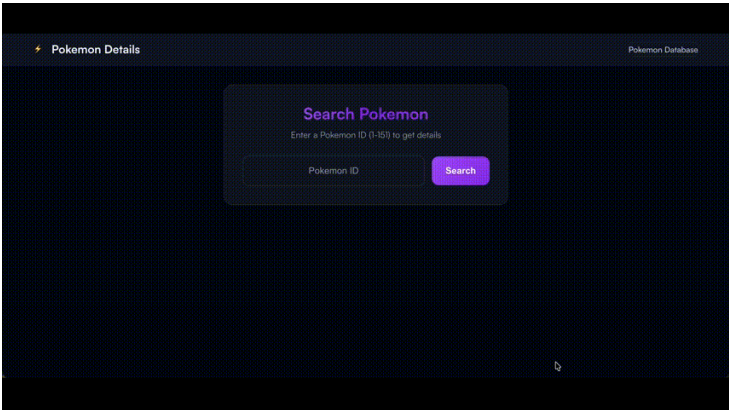
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The only file that should be modified by the candidate is "src/components/currency-converter/index.tsx". It is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 5

React(TypeScript): Pokemon Details

Create a React application that searches Pokémon by ID, shows the selected Pokémon’s details, and lets users navigate to its previous and next evolutions.

[Hide animation](#) [Show animation](#)



Your Users Should be able to:

- Enter a Pokémon ID in a **number** input field.
- Validate the ID range:
 - If the ID is not between **1** and **151**, show an alert `"Pokemon ID must be between 1 and 151"`.
- On a valid ID, make a **GET** request to `https://jsonmock.hackerrank.com/api/pokemon?id=<id>`
 - Example: ID `1` calls `https://jsonmock.hackerrank.com/api/pokemon?id=1`.
 - For an API call with a valid pokemon ID, the response will be:

```
{
  "data": {
    "id": 1,
```

```

    "num": "001",
    "name": "Bulbasaur",
    "type": [
      "Grass",
      "Poison"
    ],
    "height": "0.71 m",
    "weight": "6.9 kg",
    "candy": "Bulbasaur Candy",
    "candy_count": 25,
    "egg": "2 km",
    "spawn_chance": 0.69,
    "avg_spawns": 69,
    "spawn_time": "20:00",
    "multipliers": [1.58],
    "weaknesses": [
      "Fire",
      "Ice",
      "Flying",
      "Psychic"
    ],
    "next_evolution": [
      {
        "num": "002",
        "name": "Ivysaur"
      },
      {
        "num": "003",
        "name": "Venusaur"
      }
    ]
  }
}

```

- See the Pokémon details rendered by the Pokemon.tsx component, which receives the fetched Pokémon object as props.
- Use Next Evolution and Prev Evolution buttons based on the fetched data:
- Enable Next Evolution only if a next evolution exists.
- Enable Prev Evolution only if a previous evolution exists.
- Click Next Evolution to fetch and load the next evolved Pokémon from the API.
- Click Prev Evolution to fetch and load the previous form from the API.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is "/src/components/pokemon-details/PokemonDetails.tsx", "src/components/pokemon/Pokemon.tsx" and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.

Question - 1

React(TypeScript): Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)

 Customer Search

Name	Age	Location	Gender	Income
Jeremy Clarke	21	Seattle	Male	\$120,000
Matty Bing	25	Florida	Female	\$950,000
Philip Anderson	18	New York City	Female	\$150,000
John Smith	25	Philadelphia	Male	\$200,000
Adam Gilly	32	Denver	Male	\$2,200,000
Erica Edwards	25	Portland	Female	\$2,200,000

The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
  income: "$120,000"
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
-----------	-----------

search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

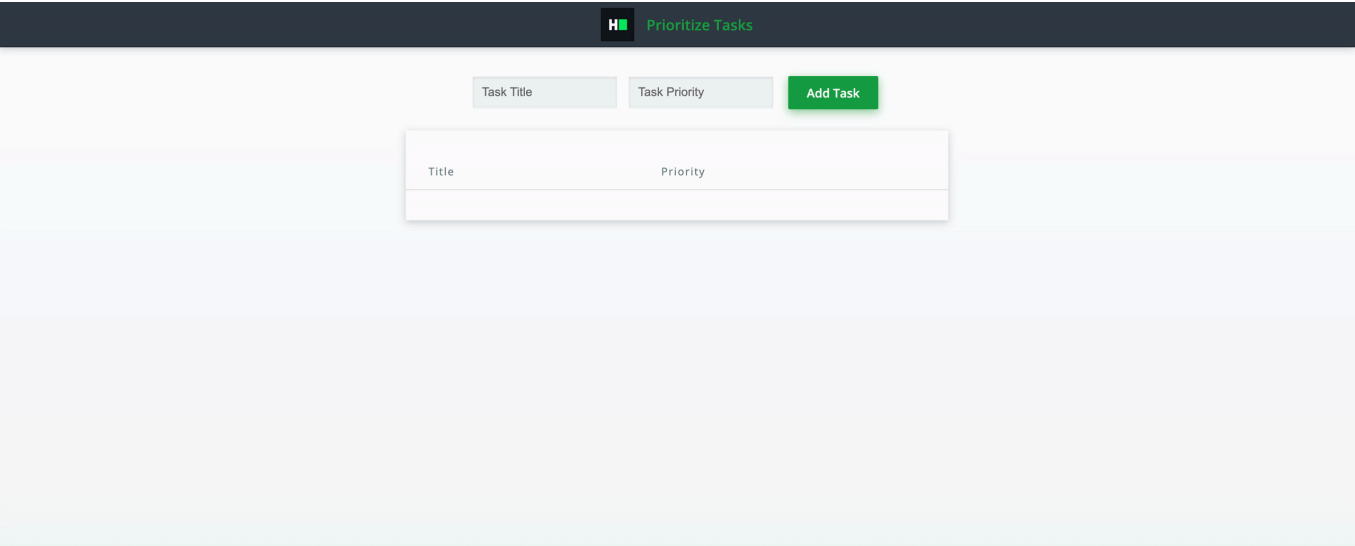
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are `src/components/CustomerList.tsx` and `src/components/SearchCustomer.tsx`. They are open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 2

React(TypeScript): Prioritize Tasks

Complete the component as shown to pass the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only one component, `Tasks`, where all the functionalities will be implemented.

- The component must have the following functionalities:
- The type of input for the:
 - Task Name input box should be "text".
 - Task Priority input box should be "number".
 - The initial view should have no tasks in the table.
 - Clicking the `Add` button should:
 - add a task and its priority in the table and sort the tasks according to priority.
 - display an alert saying "Please enter both title and task priority" if any input boxes are empty.
 - reset the input boxes to empty after adding a valid task to the table.
 - Clicking the `Delete` button should delete the corresponding task from the table.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
input-task-name	Input box for task name
input-task-priority	Input box for task priority

submit-button	Button for adding task
tasksList	List of tasks

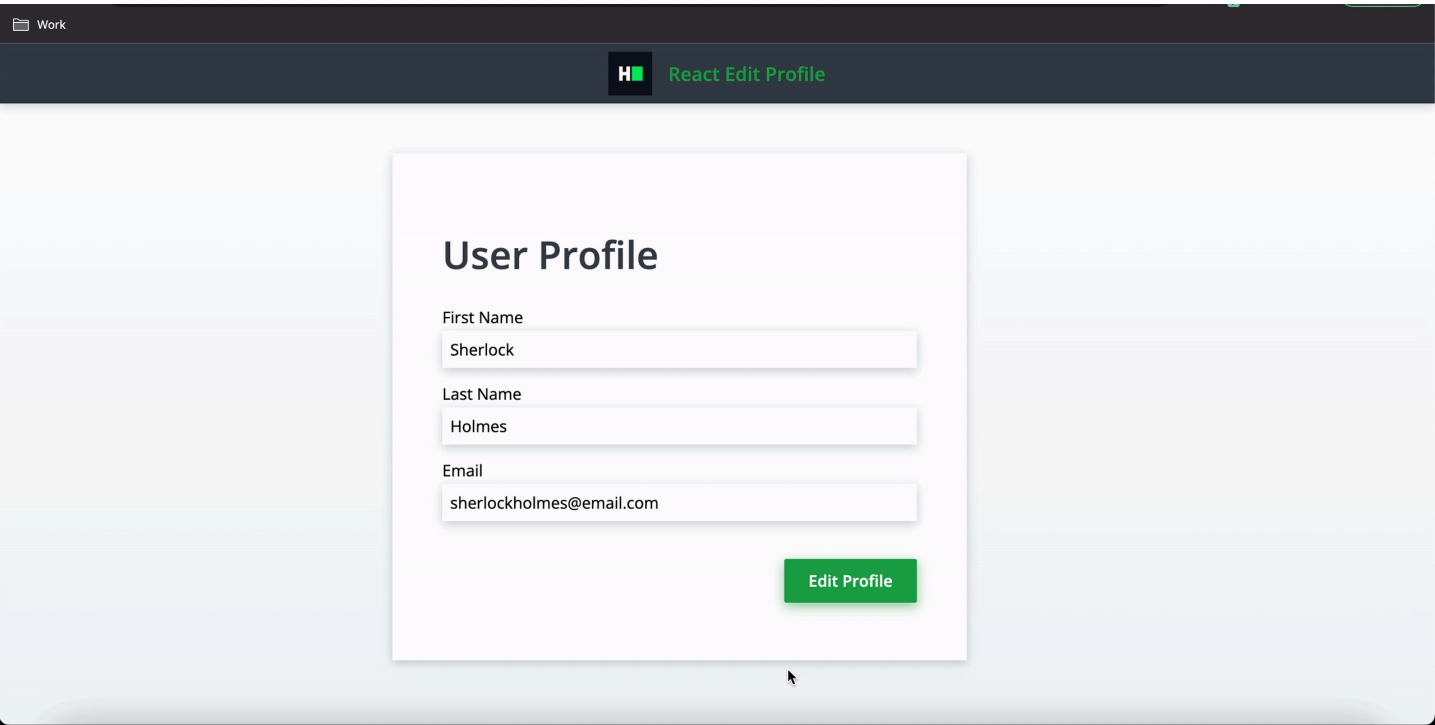
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The file that should be modified by the candidate `src/components/Books/index.tsx` is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 3

React(TypeScript): Edit Profile

Complete a React Edit Profile application as shown in order to pass all the unit tests. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only 1 component:

- The Profile component enables the user to update his/her details.

The application has the following functionalities:

- The Profile component shows the user their details and allows editing.
- Initially, there should be 3 divs with the following values:
 - First Name: **Sherlock**
 - Last Name: **Holmes**
 - Email: **sherlockholmes@email.com**
- Initially, the button should have the text value Edit Profile.
- The following functionality should be implemented when the user clicks the 'Edit Profile' button.
 - The user can now edit the details in the divs.
 - The button should now have the text value "Save Changes".
- The following functionality should be implemented when the user edits the details and clicks the Save Changes button.
 - If one or more of the fields is empty, there should be an alert prompt with the text "Please enter all profile fields" and no values should be updated.
 - If all the fields are non-empty, there should be an alert prompt with the text "Profile updated successfully".
 - The input values should be the updated values and should be in uneditable divs.
 - The button text should be Edit Profile.

The following data-testid attributes are required in the component for the tests to pass:

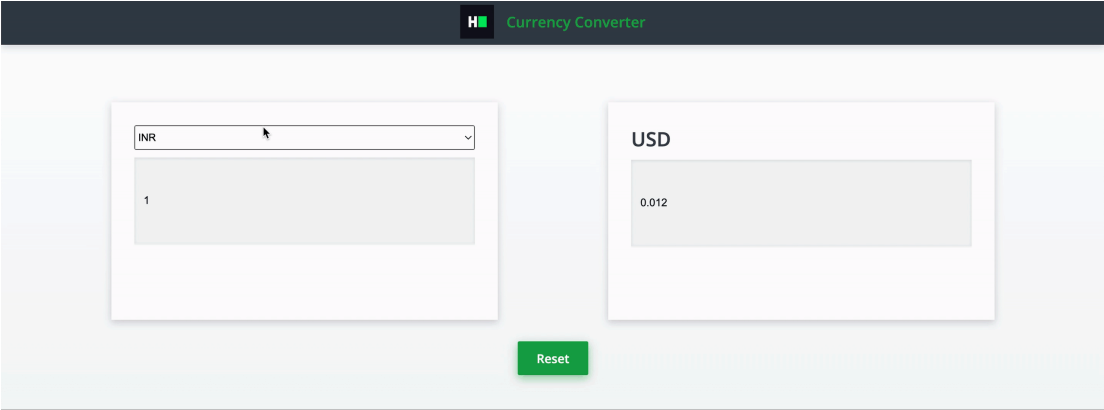
Description	Name
First Name <div>	firstNameDiv
Last Name <div>	lastNameDiv
Email <div>	emailDiv
First Name <input>	firstNameInput
Last Name <input>	lastNameInput
Email <input>	emailInput
<button>.	changeButton

The component has data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 4
React(TypeScript): Currency Converter

Complete the currency converter component that converts the value in any of the given currencies to USD as shown.

[Hide animation](#) [Show animation](#)



The list of available currencies is imported as "exchangeRates" in the component that can be used to render the list of currencies. The initial values for the currency and the input box are also imported from "exchangeRates.ts".

It is given that:

- The first input box is to be used to take input.
- The second input box is read-only to display the converted value.
- Only positive integer or decimal values can be taken as input.

The component must have the following functionalities:

- The default value for:
 - input box should be 1.
 - the currency to display in the select menu should be INR.
 - The second input box should be the conversion of 1 INR to USD i.e. 0.012.
- The select menu should have five currencies in the options.

- The conversion should happen automatically:
 - when there is any input change.
 - on selecting another currency.
- The converted value should be up to 3 decimal places.
- The placeholder text should change automatically according to the currency.
- The Reset button should reset the input to 1 and give the conversion in the current currency.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
select menu	select-currency
first input box	input-value
second input box	converted-value
Clear Input button	clear-value

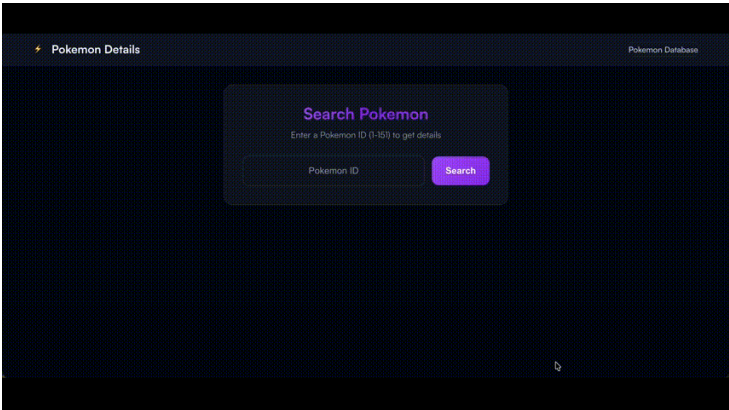
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The only file that should be modified by the candidate is "src/components/currency-converter/index.tsx". It is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 5

React(TypeScript): Pokemon Details

Create a React application that searches Pokémon by ID, shows the selected Pokémon’s details, and lets users navigate to its previous and next evolutions.

[Hide animation](#) [Show animation](#)



Your Users Should be able to:

- Enter a Pokémon ID in a **number** input field.
- Validate the ID range:
 - If the ID is not between **1** and **151**, show an alert `"Pokemon ID must be between 1 and 151"`.
- On a valid ID, make a **GET** request to `https://jsonmock.hackerrank.com/api/pokemon?id=<id>`
 - Example: ID `1` calls `https://jsonmock.hackerrank.com/api/pokemon?id=1`.
 - For an API call with a valid pokemon ID, the response will be:

```
{
  "data": {
    "id": 1,
```

```

    "num": "001",
    "name": "Bulbasaur",
    "type": [
      "Grass",
      "Poison"
    ],
    "height": "0.71 m",
    "weight": "6.9 kg",
    "candy": "Bulbasaur Candy",
    "candy_count": 25,
    "egg": "2 km",
    "spawn_chance": 0.69,
    "avg_spawns": 69,
    "spawn_time": "20:00",
    "multipliers": [1.58],
    "weaknesses": [
      "Fire",
      "Ice",
      "Flying",
      "Psychic"
    ],
    "next_evolution": [
      {
        "num": "002",
        "name": "Ivysaur"
      },
      {
        "num": "003",
        "name": "Venusaur"
      }
    ]
  }
}

```

- See the Pokémon details rendered by the Pokemon.tsx component, which receives the fetched Pokémon object as props.
- Use Next Evolution and Prev Evolution buttons based on the fetched data:
- Enable Next Evolution only if a next evolution exists.
- Enable Prev Evolution only if a previous evolution exists.
- Click Next Evolution to fetch and load the next evolved Pokémon from the API.
- Click Prev Evolution to fetch and load the previous form from the API.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is "/src/components/pokemon-details/PokemonDetails.tsx", "src/components/pokemon/Pokemon.tsx" and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.

Question - 1

React(TypeScript): Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)

 Customer Search

Name	Age	Location	Gender	Income
Jeremy Clarke	21	Seattle	Male	\$120,000
Matty Bing	25	Florida	Female	\$950,000
Philip Anderson	18	New York City	Female	\$150,000
John Smith	25	Philadelphia	Male	\$200,000
Adam Gilly	32	Denver	Male	\$2,200,000
Erica Edwards	25	Portland	Female	\$2,200,000

The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
  income: "$120,000"
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
-----------	-----------

search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

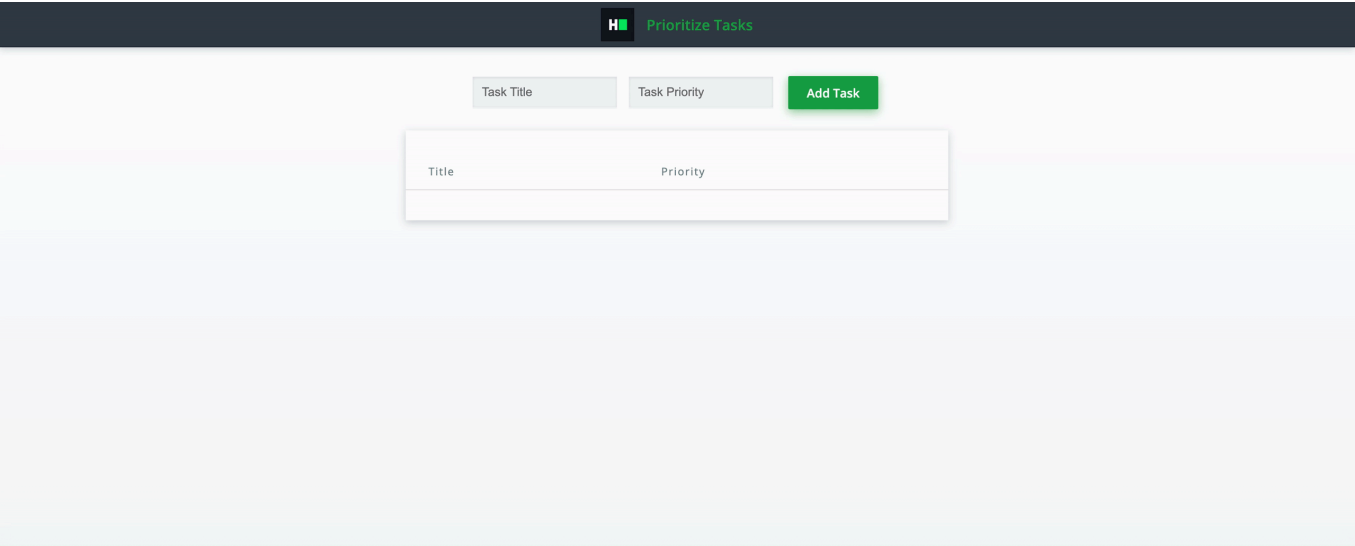
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are `src/components/CustomerList.tsx` and `src/components/SearchCustomer.tsx`. They are open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 2

React(TypeScript): Prioritize Tasks

Complete the component as shown to pass the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only one component, `Tasks`, where all the functionalities will be implemented.

- The component must have the following functionalities:
- The type of input for the:
 - Task Name input box should be "text".
 - Task Priority input box should be "number".
 - The initial view should have no tasks in the table.
 - Clicking the `Add` button should:
 - add a task and its priority in the table and sort the tasks according to priority.
 - display an alert saying "Please enter both title and task priority" if any input boxes are empty.
 - reset the input boxes to empty after adding a valid task to the table.
 - Clicking the `Delete` button should delete the corresponding task from the table.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
input-task-name	Input box for task name
input-task-priority	Input box for task priority

submit-button	Button for adding task
tasksList	List of tasks

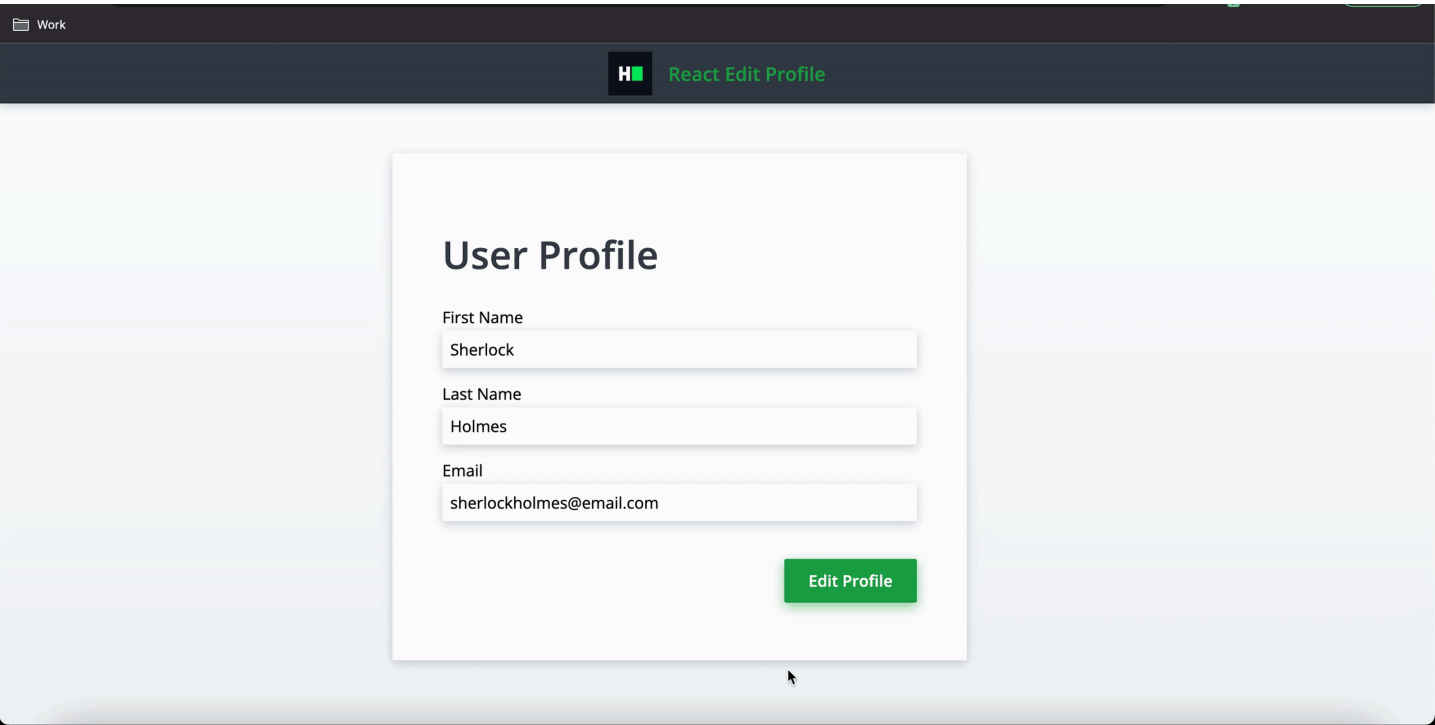
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The file that should be modified by the candidate `src/components/Books/index.tsx` is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 3

React(TypeScript): Edit Profile

Complete a React Edit Profile application as shown in order to pass all the unit tests. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only 1 component:

- The Profile component enables the user to update his/her details.

The application has the following functionalities:

- The Profile component shows the user their details and allows editing.
- Initially, there should be 3 divs with the following values:
 - First Name: **Sherlock**
 - Last Name: **Holmes**
 - Email: **sherlockholmes@email.com**
- Initially, the button should have the text value Edit Profile.
- The following functionality should be implemented when the user clicks the 'Edit Profile' button.
 - The user can now edit the details in the divs.
 - The button should now have the text value "Save Changes".
- The following functionality should be implemented when the user edits the details and clicks the Save Changes button.
 - If one or more of the fields is empty, there should be an alert prompt with the text "Please enter all profile fields" and no values should be updated.
 - If all the fields are non-empty, there should be an alert prompt with the text "Profile updated successfully".
 - The input values should be the updated values and should be in uneditable divs.
 - The button text should be Edit Profile.

The following data-testid attributes are required in the component for the tests to pass:

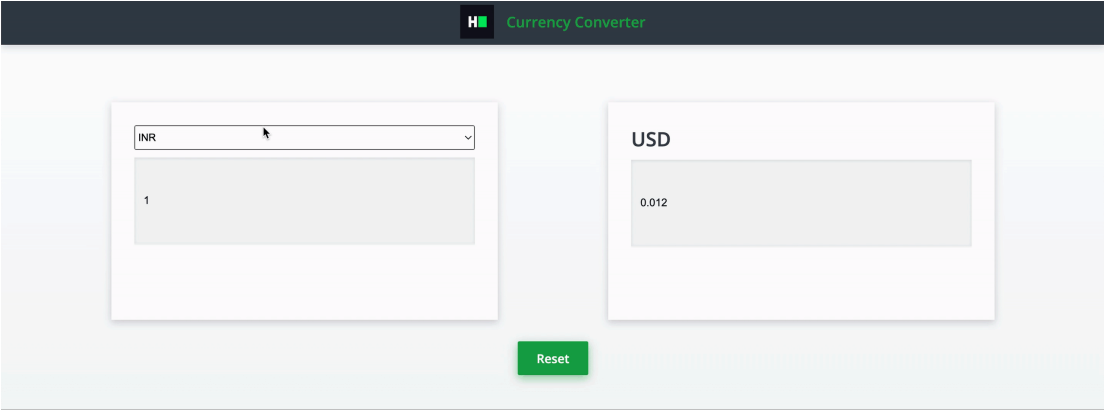
Description	Name
First Name <div>	firstNameDiv
Last Name <div>	lastNameDiv
Email <div>	emailDiv
First Name <input>	firstNameInput
Last Name <input>	lastNameInput
Email <input>	emailInput
<button>.	changeButton

The component has data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 4
React(TypeScript): Currency Converter

Complete the currency converter component that converts the value in any of the given currencies to USD as shown.

[Hide animation](#) [Show animation](#)



The list of available currencies is imported as "exchangeRates" in the component that can be used to render the list of currencies. The initial values for the currency and the input box are also imported from "exchangeRates.ts".

It is given that:

- The first input box is to be used to take input.
- The second input box is read-only to display the converted value.
- Only positive integer or decimal values can be taken as input.

The component must have the following functionalities:

- The default value for:
 - input box should be 1.
 - the currency to display in the select menu should be INR.
 - The second input box should be the conversion of 1 INR to USD i.e. 0.012.
- The select menu should have five currencies in the options.

- The conversion should happen automatically:
 - when there is any input change.
 - on selecting another currency.
- The converted value should be up to 3 decimal places.
- The placeholder text should change automatically according to the currency.
- The Reset button should reset the input to 1 and give the conversion in the current currency.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
select menu	select-currency
first input box	input-value
second input box	converted-value
Clear Input button	clear-value

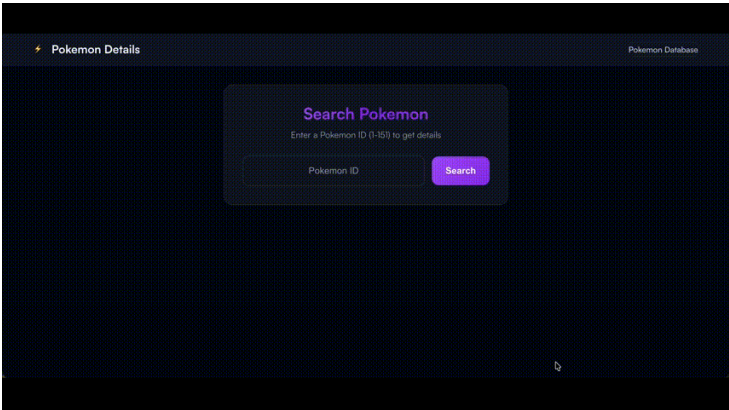
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The only file that should be modified by the candidate is "src/components/currency-converter/index.tsx". It is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 5

React(TypeScript): Pokemon Details

Create a React application that searches Pokémon by ID, shows the selected Pokémon’s details, and lets users navigate to its previous and next evolutions.

[Hide animation](#) [Show animation](#)



Your Users Should be able to:

- Enter a Pokémon ID in a **number** input field.
- Validate the ID range:
 - If the ID is not between **1** and **151**, show an alert `"Pokemon ID must be between 1 and 151"`.
- On a valid ID, make a **GET** request to `https://jsonmock.hackerrank.com/api/pokemon?id=<id>`
 - Example: ID **1** calls `https://jsonmock.hackerrank.com/api/pokemon?id=1`.
 - For an API call with a valid pokemon ID, the response will be:

```
{
  "data": {
    "id": 1,
```

```

    "num": "001",
    "name": "Bulbasaur",
    "type": [
      "Grass",
      "Poison"
    ],
    "height": "0.71 m",
    "weight": "6.9 kg",
    "candy": "Bulbasaur Candy",
    "candy_count": 25,
    "egg": "2 km",
    "spawn_chance": 0.69,
    "avg_spawns": 69,
    "spawn_time": "20:00",
    "multipliers": [1.58],
    "weaknesses": [
      "Fire",
      "Ice",
      "Flying",
      "Psychic"
    ],
    "next_evolution": [
      {
        "num": "002",
        "name": "Ivysaur"
      },
      {
        "num": "003",
        "name": "Venusaur"
      }
    ]
  }
}

```

- See the Pokémon details rendered by the Pokemon.tsx component, which receives the fetched Pokémon object as props.
- Use Next Evolution and Prev Evolution buttons based on the fetched data:
- Enable Next Evolution only if a next evolution exists.
- Enable Prev Evolution only if a previous evolution exists.
- Click Next Evolution to fetch and load the next evolved Pokémon from the API.
- Click Prev Evolution to fetch and load the previous form from the API.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is "/src/components/pokemon-details/PokemonDetails.tsx", "src/components/pokemon/Pokemon.tsx" and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.

Question - 1

React(TypeScript): Customer Search

Complete the component as shown to pass all the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)

 Customer Search

Name	Age	Location	Gender	Income
Jeremy Clarke	21	Seattle	Male	\$120,000
Matty Bing	25	Florida	Female	\$950,000
Philip Anderson	18	New York City	Female	\$150,000
John Smith	25	Philadelphia	Male	\$200,000
Adam Gilly	32	Denver	Male	\$2,200,000
Erica Edwards	25	Portland	Female	\$2,200,000

The list of available customers and their details is imported as `List` in the `SearchCustomer` component. Use the list to render them as shown.

The data in `List` file is in this form.

```
{
  name: "Jeremy Clarke",
  age: 21,
  location: "Seattle",
  gender: "Male",
  income: "$120,000"
}
```

The application has 2 components:

- `SearchCustomer` - contains and search box and `CustomerList` component.
- `CustomerList` - displays the details of the customer based on the search term in a table.

The component must have the following functionalities:

- Initially, the input field must be empty. Whenever the input is empty, all the customers passed in the input to the component must be rendered in the list.
- The type of input in the input box should be `text`.
- As soon as the search term is typed in the input, search for customer records with any field that starts with the search term. For example, if the search term is "Phil", then records with the name "Philip Anderson" and the location "Philadelphia" should be displayed in the results.
- The filtered list should preserve the order customers are given in the input to the component.
- If the search term returns no customers, do not render the table. Instead, display the message "No Results Found!".
- The search results should be case-sensitive.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
-----------	-----------

search-input	Search Input Box
searched-customers	List of searched customers
no-results	div when no customer matches the search term

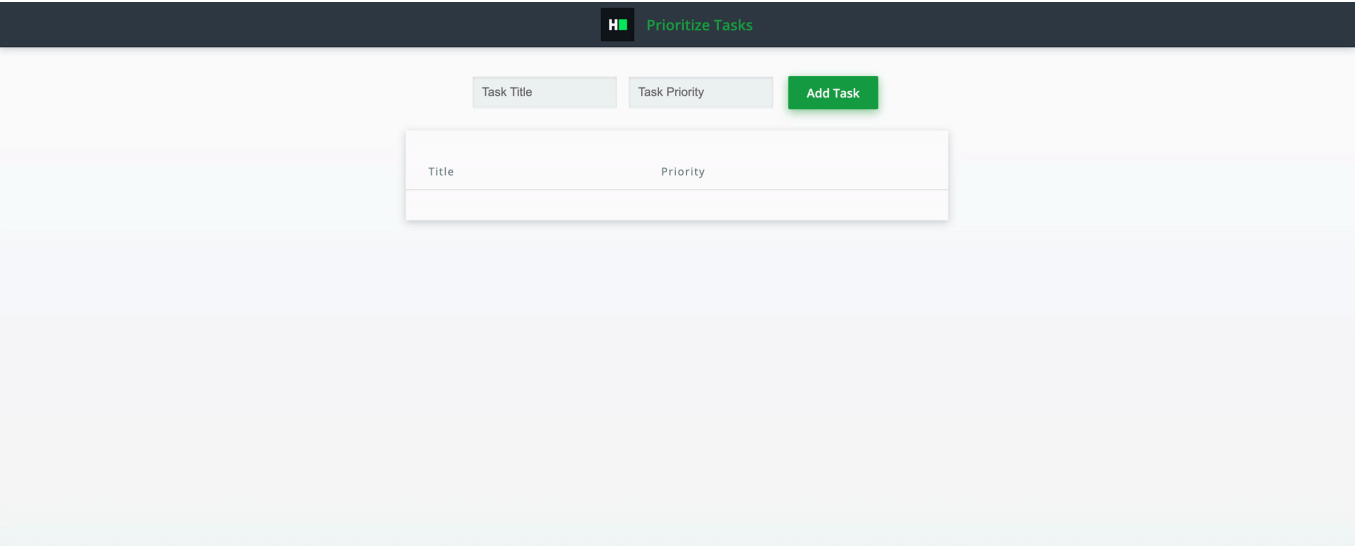
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The files that should be modified by the candidate are `src/components/CustomerList.tsx` and `src/components/SearchCustomer.tsx`. They are open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 2

React(TypeScript): Prioritize Tasks

Complete the component as shown to pass the test cases. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only one component, `Tasks`, where all the functionalities will be implemented.

- The component must have the following functionalities:
- The type of input for the:
 - Task Name input box should be "text".
 - Task Priority input box should be "number".
 - The initial view should have no tasks in the table.
 - Clicking the `Add` button should:
 - add a task and its priority in the table and sort the tasks according to priority.
 - display an alert saying "Please enter both title and task priority" if any input boxes are empty.
 - reset the input boxes to empty after adding a valid task to the table.
 - Clicking the `Delete` button should delete the corresponding task from the table.

The following data-testid attributes are required in the component for the tests to pass:

Attribute	Component
input-task-name	Input box for task name
input-task-priority	Input box for task priority

submit-button	Button for adding task
tasksList	List of tasks

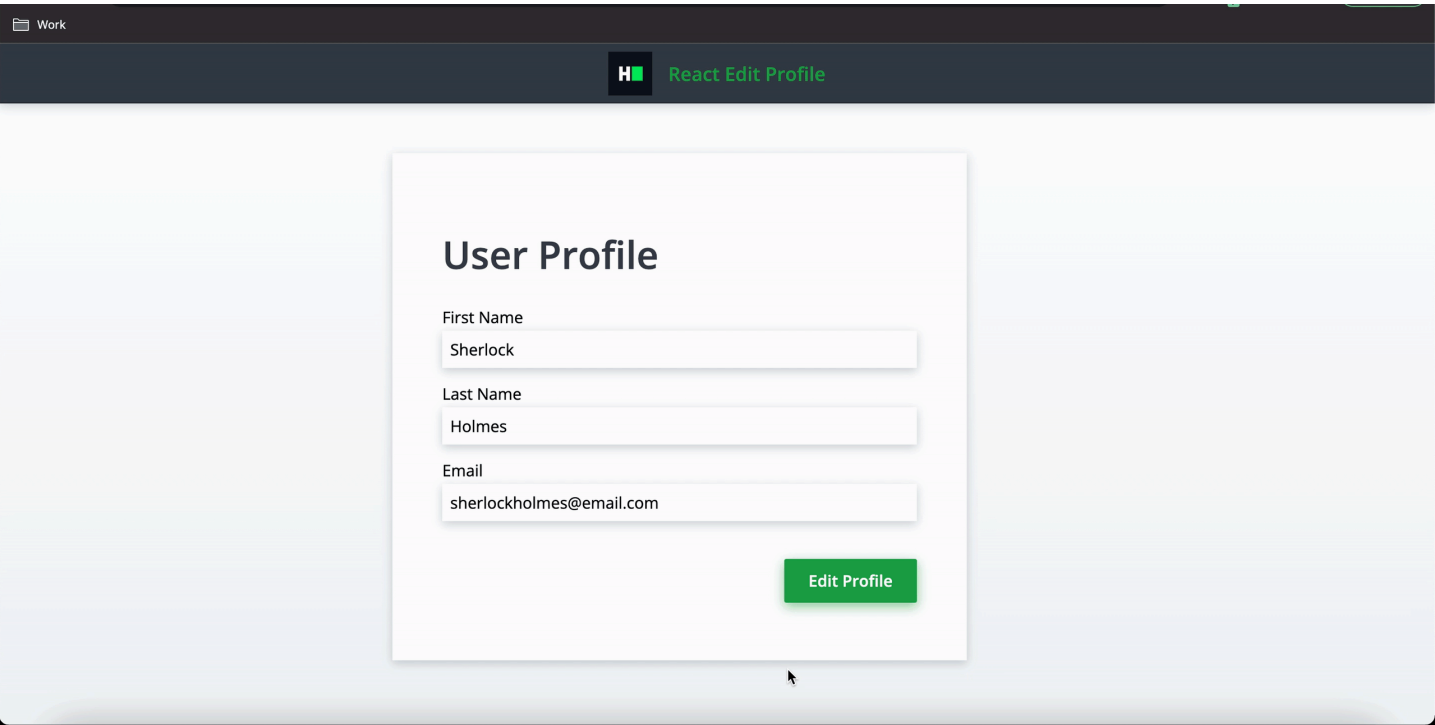
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The file that should be modified by the candidate `src/components/Books/index.tsx` is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 3

React(TypeScript): Edit Profile

Complete a React Edit Profile application as shown in order to pass all the unit tests. Certain core React functionalities are already implemented.

[Hide animation](#) [Show animation](#)



The application has only 1 component:

- The Profile component enables the user to update his/her details.

The application has the following functionalities:

- The Profile component shows the user their details and allows editing.
- Initially, there should be 3 divs with the following values:
 - First Name: **Sherlock**
 - Last Name: **Holmes**
 - Email: **sherlockholmes@email.com**
- Initially, the button should have the text value Edit Profile.
- The following functionality should be implemented when the user clicks the 'Edit Profile' button.
 - The user can now edit the details in the divs.
 - The button should now have the text value "Save Changes".
- The following functionality should be implemented when the user edits the details and clicks the Save Changes button.
 - If one or more of the fields is empty, there should be an alert prompt with the text "Please enter all profile fields" and no values should be updated.
 - If all the fields are non-empty, there should be an alert prompt with the text "Profile updated successfully".
 - The input values should be the updated values and should be in uneditable divs.
 - The button text should be Edit Profile.

The following data-testid attributes are required in the component for the tests to pass:

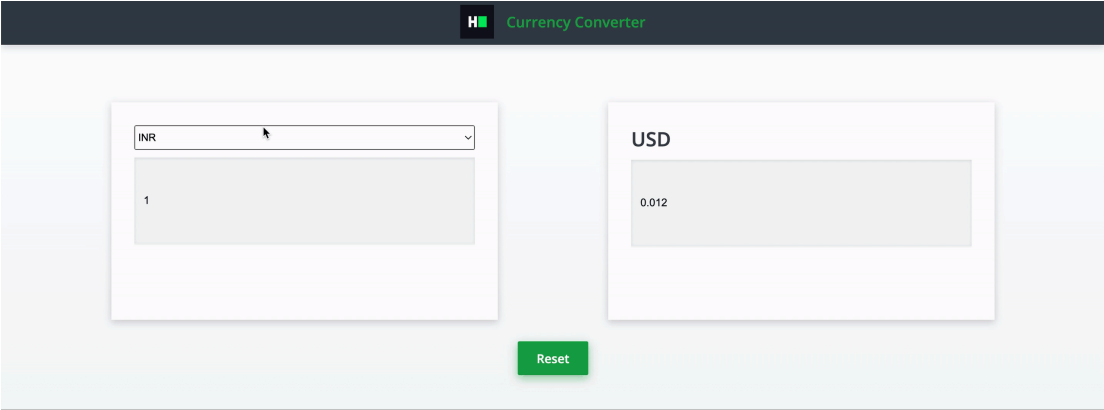
Description	Name
First Name <div>	firstNameDiv
Last Name <div>	lastNameDiv
Email <div>	emailDiv
First Name <input>	firstNameInput
Last Name <input>	lastNameInput
Email <input>	emailInput
<button>.	changeButton

The component has data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.

Question - 4
React(TypeScript): Currency Converter

Complete the currency converter component that converts the value in any of the given currencies to USD as shown.

[Hide animation](#) [Show animation](#)



The list of available currencies is imported as "exchangeRates" in the component that can be used to render the list of currencies. The initial values for the currency and the input box are also imported from "exchangeRates.ts".

It is given that:

- The first input box is to be used to take input.
- The second input box is read-only to display the converted value.
- Only positive integer or decimal values can be taken as input.

The component must have the following functionalities:

- The default value for:
 - input box should be 1.
 - the currency to display in the select menu should be INR.
 - The second input box should be the conversion of 1 INR to USD i.e. 0.012.
- The select menu should have five currencies in the options.

- The conversion should happen automatically:
 - when there is any input change.
 - on selecting another currency.
- The converted value should be up to 3 decimal places.
- The placeholder text should change automatically according to the currency.
- The Reset button should reset the input to 1 and give the conversion in the current currency.

The following data-testid attributes are required in the component for the tests to pass:

Component	Attribute
select menu	select-currency
first input box	input-value
second input box	converted-value
Clear Input button	clear-value

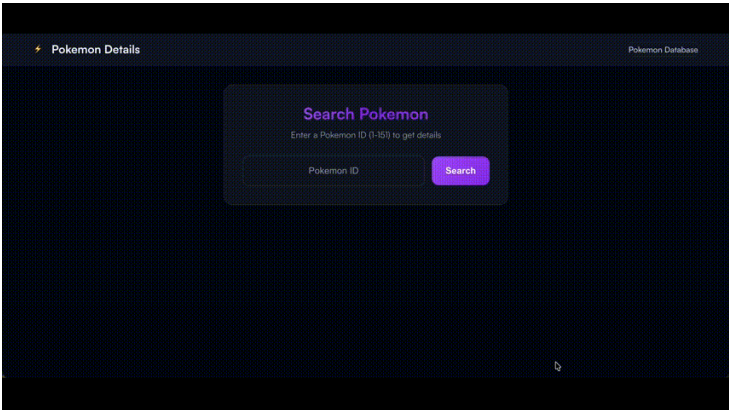
- Note:
- Components have data-testid attributes for test cases and certain classes and ids for rendering purposes. They should not be changed.
 - The only file that should be modified by the candidate is "src/components/currency-converter/index.tsx". It is open by default in the system editor.
 - Avoid making changes to other files in the project structure.

Question - 5

React(TypeScript): Pokemon Details

Create a React application that searches Pokémon by ID, shows the selected Pokémon’s details, and lets users navigate to its previous and next evolutions.

[Hide animation](#) [Show animation](#)



Your Users Should be able to:

- Enter a Pokémon ID in a **number** input field.
- Validate the ID range:
 - If the ID is not between **1** and **151**, show an alert `"Pokemon ID must be between 1 and 151"`.
- On a valid ID, make a **GET** request to `https://jsonmock.hackerrank.com/api/pokemon?id=<id>`
 - Example: ID **1** calls `https://jsonmock.hackerrank.com/api/pokemon?id=1`.
 - For an API call with a valid pokemon ID, the response will be:

```
{
  "data": {
    "id": 1,
```

```

    "num": "001",
    "name": "Bulbasaur",
    "type": [
      "Grass",
      "Poison"
    ],
    "height": "0.71 m",
    "weight": "6.9 kg",
    "candy": "Bulbasaur Candy",
    "candy_count": 25,
    "egg": "2 km",
    "spawn_chance": 0.69,
    "avg_spawns": 69,
    "spawn_time": "20:00",
    "multipliers": [1.58],
    "weaknesses": [
      "Fire",
      "Ice",
      "Flying",
      "Psychic"
    ],
    "next_evolution": [
      {
        "num": "002",
        "name": "Ivysaur"
      },
      {
        "num": "003",
        "name": "Venusaur"
      }
    ]
  }
}

```

- See the Pokémon details rendered by the Pokemon.tsx component, which receives the fetched Pokémon object as props.
- Use Next Evolution and Prev Evolution buttons based on the fetched data:
- Enable Next Evolution only if a next evolution exists.
- Enable Prev Evolution only if a previous evolution exists.
- Click Next Evolution to fetch and load the next evolved Pokémon from the API.
- Click Prev Evolution to fetch and load the previous form from the API.

Note:

- Please note that components have the above data-testid attributes for test cases and certain classes and ids for rendering purposes. It is advised not to change them.
- The only file that should be modified by the candidate is "/src/components/pokemon-details/PokemonDetails.tsx", "src/components/pokemon/Pokemon.tsx" and is open by default by the system editor.
- It is advised for candidates to avoid making any changes to the rest of the files in the project structure.